European Journal of Operational Research 327 (2025) 875–891

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Production, Manufacturing, Transportation and Logistics

Staggered routing in autonomous mobility-on-demand systems

Antonio Coppola [a,*], Gerhard Hiermann [a,b], Dario Paccagnan [c], Maximilian Schiffer [d]

[a] School of Management, Technical University of Munich, Germany
[b] Institute of Production and Logistics Management, Johannes Kepler University Linz, Austria
[c] Department of Computing, Imperial College London, UK
[d] School of Management & Munich Data Science Institute, Technical University of Munich, Germany

ARTICLE INFO

Keywords:
Routing
Autonomous mobility-on-demand
Trip-staggering

ABSTRACT

In autonomous mobility-on-demand systems, effectively managing vehicle flows to mitigate induced congestion and ensure efficient operations is imperative for system performance and positive customer experience. Against this background, we study the potential of staggered routing, i.e., purposely delaying trip departures from a system perspective, in order to reduce congestion and ensure efficient operations while still meeting customer time windows. We formalize the underlying planning problem and show how to efficiently model it as a mixed integer linear program. Moreover, we present a matheuristic that allows us to efficiently solve large-scale real-world instances both in an offline full-information setting and its online rolling horizon counterpart. We conduct a numerical study for Manhattan, New York City, focusing on low- and highly-congested scenarios. Our results show that in low-congestion scenarios, staggering trip departures allows mitigating, on average, 98% of the induced congestion in a full information setting. In a rolling horizon setting, our algorithm allows us to reduce 82% of the induced congestion. In high-congestion scenarios, we observe an average reduction of 60% as the full information bound and an average reduction of 30% in our online setting. Surprisingly, we show that these reductions can be reached by shifting trip departures by a maximum of six minutes in both the low and high-congestion scenarios.

1. Introduction

Urban congestion has become an urgent challenge for modern cities, with significant economic, environmental, and public health consequences. In 2022, the average U.S. driver spent 51 h stuck in traffic, resulting in an economic loss of $81 billion (Pishue, 2023). Beyond financial costs, congestion exacerbates air pollution and greenhouse gas emissions, contributing to climate change and adverse health effects (Levy et al., 2010). A major contributor to worsening congestion is the expansion of ride-hailing services, which, rather than alleviating congestion, have been shown to displace public transportation trips and increase overall vehicle miles traveled (Agarwal et al., 2023; Tirachini & Gomez-Lobo, 2020).

Emerging mobility paradigms such as AMoD have been proposed as potential solutions to mitigate these externalities. AMoD systems consist of centrally coordinated fleets of self-driving vehicles providing ride-hailing services (Pavone, 2015). By leveraging real-time system-wide state observation, centralized dispatching, and rebalancing algorithms, AMoD systems have the potential to improve transport efficiency compared to private car travel or uncoordinated ride-hailing services. However, these benefits come with trade-offs, as AMoD services also risk exacerbating peak-hour congestion if not carefully managed. Despite their autonomous capabilities, AMoD systems fundamentally share the same operational characteristics as conventional ride-hailing fleets: they generate additional vehicle miles traveled, induce modal shifts away from public transit, and contribute to traffic demand surges during peak hours. As such, while AMoD systems offer enhanced control and efficiency mechanisms, their core impact on urban congestion dynamics remains closely aligned with that of traditional ride-hailing services.

To address these challenges, researchers and policymakers have explored various strategies to mitigate the congestion impact of ride-hailing and AMoD systems. While AMoD services offer the potential for improved fleet control, their effectiveness in reducing congestion depends on broader traffic dynamics and modal shifts. Accordingly, recent research highlights the importance of integrating AMoD services with public transportation and promoting ride-pooling as primary congestion mitigation strategies (Ruch et al., 2020; Salazar et al., 2019). In addition to these measures, congestion pricing and demand-responsive fleet management have emerged as critical tools. Congestion pricing

* Corresponding author.
E-mail addresses: antonio.coppola@tum.de (A. Coppola), gerhard.hiermann@jku.at (G. Hiermann), d.paccagnan@imperial.ac.uk (D. Paccagnan), schiffer@tum.de (M. Schiffer).

https://doi.org/10.1016/j.ejor.2025.06.008
Received 17 September 2024; Accepted 6 June 2025
Available online 19 June 2025
0377-2217/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

leverages economic incentives to redistribute traffic across time and space by charging fees for road use during peak hours (Arnott et al., 1990; Roughgarden, 2005), while demand-responsive fleet management seeks to optimize vehicle routing and dispatching to reduce system-wide inefficiencies (Jahn et al., 2005; Patriksson, 2015). In this context, fleet management and routing decisions taken by an (AMoD) operator must not necessarily align with a municipality system perspective that aims to minimize overall congestion beyond the fleet's operations. While optimizing a fleet's operations with respect to congestion effects firstly improves the customer's experience and convenience it can further allow to mitigate local congestion bottlenecks. In fact, targeted interventions at the fleet level can contribute to reducing congestion at the system level, even if only applied to a subset of the traffic network (Battifarano & Qian, 2023).

Despite extensive research into congestion pricing and fleet optimization, many existing methods predominantly focus on distributing traffic across space rather than time (Jalota et al., 2023). While some demand management strategies incorporate departure delays to alleviate peak-load congestion, their application to individual fleet operators remains underexplored. AMoD systems, with their centralized control, offer an opportunity to implement time-based congestion mitigation strategies at a finer level of granularity.

Against these backdrops, we introduce the concept of *staggered routing*, where an AMoD operator delays the departure of trips over time to reduce local congestion bottlenecks while ensuring all trips' drop-off deadlines are met. Specifically, we take a first step to unravel the *potential* of staggered routing in AMoD systems. This approach complements existing congestion mitigation efforts by leveraging AMoD-specific control capabilities to improve system efficiency while reducing the fleet's negative externalities.

1.1. Contribution

Our work aims to inform on the impact of staggering trip departures in reducing congestion. Towards this goal, we present three key contributions. First, we introduce the problem of *staggered routing*, i.e., the problem of staggering the fleet departure times so as to minimize the resulting congestion while meeting drop-off deadlines. We then formulate this problem as a mixed integer linear program (MILP) and show that it is NP-hard. Second, we provide an effective algorithmic framework to solve the staggered routing problem at scale. Specifically, we (i) show how a large number of the resulting big-M constraints can be dropped without altering the optimal solutions, and (ii) develop a matheuristic that allows us to solve large-scale instances. This matheuristic bases on a construction heuristic, a problem-specific local search for intensification, while relying on our MILP to escape local optima and verify optimality. Finally, we apply our methodology to a real-world case study for the Manhattan area in New York City. Specifically, we study the impact that staggered routing has in both an offline and an online decision-making setting.

Our results show that in LC scenarios, i.e., scenarios in which only the AMoD fleet induces congestion but the system without the fleet would remain uncongested, staggering trip departures allow to mitigate on average 98% of the induced congestion in a full-information setting. While not entirely reaching this upper bound, our rolling horizon approach still allows us to reduce 82% of the induced congestion in the online LC scenario. In HC scenarios, i.e., scenarios in which the system already encounters exogenous congestion and the AMoD fleet induces further congestion, we observe an average congestion reduction of 60% and 30% in the full-information and rolling-horizon settings. Surprisingly, we show that these reductions can be reached by shifting trip departures of a maximum of two minutes in the LC scenario and by six minutes in the HC scenario while meeting each trip's arrival time.

1.2. Technical challenges and our approach

Solving the staggered routing problem at scale requires tackling at least three intertwined challenges.

First, following the commonly employed approach whereby one keeps track of the congestion level on each arc at predetermined time intervals (see, e.g., time-expanded construction in Köhler et al. (2002)) is simply not possible as the size of the resulting optimization problem grows too quickly, making the solution of even moderately sized instances beyond reach. We tackle this challenge by taking a so-called Lagrangian perspective, that is, we "follow" each trip along its path and only keep track of the congestion this trip encounters at the time when it enters each arc on its path. Unlike the Eulerian view, which focuses on the congestion at a fixed location in the network and considers the total flow passing through that location over time, the Lagrangian perspective tracks individual trips as they traverse the network (Bennett, 2006). This shift allows us to formalize an optimization problem that is significantly more compact.

Second, while the Lagrangian perspective we adopt allows us to reduce the problem dimensionality, it makes it more difficult to determine the congestion level each trip encounters along the traveled arcs, leading to a set of non-linear constraints. Although easily handled through big-M reformulations, the number of such constraints grows quickly such that the resulting continuous relaxation performs poorly for large problem instances. We resolve this challenge by showing how to significantly reduce their number without affecting optimality. We do so by carefully reformulating the problem over a multigraph.

Third, the resulting MILP can still not be solved at scale by state-of-the-art solvers. We tackle this challenge by developing a local-search algorithm that we embed within the branch and bound tree generated when solving the MILP. Specifically, whenever we find a new incumbent by solving the MILP, we stop and improve this solution through our local search before feeding it back to warmstart the MILP. While our local search follows a greedy approach based on delay severity, its development requires careful considerations as efficiently evaluating the impact that modifying one departure time has on all other trips is nontrivial.

Complimentary to this, scaling our solution to complex real-world networks introduces additional computational challenges due to the large number of road segments involved. By utilizing contraction hierarchies (cf. Geisberger et al., 2008), we reduce the network's complexity while preserving shortest path lengths. This strategy significantly improves the model's applicability and computational efficiency, making it suitable for studying staggered routing in real-world scenarios.

1.3. Related work

Our work connects with various streams of literature for the optimization of AMoD systems, an area that has recently attracted significant attention. Given the sheer volume of literature in this space, providing a comprehensive overview is beyond the scope of this work, and we refer the interested reader to Narayanan et al. (2020) and Zardini et al. (2022).

While our focus is on operational fleet control, we briefly reference strategic models for completeness. These studies typically use time-invariant network flow formulations to assess the broader impact of AMoD trips on traffic (see, e.g., Rossi et al., 2018; Salazar et al., 2019) and guide long-term decisions such as fleet sizing and infrastructure planning. However, since these models operate in steady-state regimes, they do not account for dynamic routing decisions, which are central to our study.

Existing *operational* control approaches are instead based on fine-grained models that do incorporate a time dimension as they aim at devising an online control policy. Several works have been published in this domain, focusing on planning tasks such as vehicle-to-request dispatching, vehicle rebalancing, and request pooling. In this context, various methodological approaches have been studied, ranging from model predictive control for vehicle dispatching (see, e.g., Iglesias et al., 2018; Liu & Samaranayake, 2022; Tsao et al., 2018) and request pooling (see, e.g., Alonso-Mora et al., 2017), to deep reinforcement

learning for rebalancing (see, e.g., Gammelli et al., 2021; Jiao et al., 2021; Liang et al., 2021; Skordilis et al., 2021) and dispatching (see, e.g., Enders et al., 2023; Li et al., 2019; Eshkevari et al., 2022; Tang et al., 2019; Xu et al., 2018; Zhou et al., 2019), to optimization-augmented learning pipelines (see, e.g., Jungel et al., 2023; Zhang et al., 2017). However, these approaches focus on optimizing the routing decisions over space but do not focus on the possibility of staggering the departures over time, which is the focus of this work. Surprisingly, even studies that explicitly model congestion are scarce as existing research often bases travel times on precise forecasts or deterministic assumptions.

Delaying trip departures as a demand management strategy has been extensively studied to optimize vehicle travel times within street networks. Traditional methods, such as dynamic traffic assignment and route guidance schemes, have utilized complex microscopic models to address congestion, but these become cumbersome in large-scale applications (Chabini, 1998; Daganzo, 2007). More recent approaches have leveraged aggregated traffic models like the macroscopic fundamental diagram (MFD), which are favored for their scalability (Ramezani et al., 2015). Applications include maximizing trip completion rate across all regions of a multi-regional traffic network and addressing on-time arrivals to minimize the discrepancy between drivers' desired and actual arrival time (Menelaou et al., 2022, 2024). Aggregated accumulation-based approaches have also been coupled with more granular trip-based MFD models that describe individual trip details post-departure time adjustments (Kumarage et al., 2021; Yildirimoglu & Ramezani, 2020). However, these models typically operate at a *system-wide* level. In contrast, a fleet-based approach to managing traffic with staggered time policies has not been explored due to the lack of control of individual vehicle flows prior to the advent of AMoD systems, which we aim to pioneer in this work.

Finally, the MILP framework with arc-based time windows developed in this work shares similar foundational principles with both the core times notion in resource-constrained project scheduling (RCPSP) and the constraint propagation principles used in constraint programming (CP). In RCPSP, core times refer to intervals during which each activity must be active in any valid schedule, reflecting the earliest and latest times activities can start or finish under resource and precedence constraints (Bartusch et al., 1988; Neumann et al., 2002). Similarly, in CP-based scheduling, constraints on activities' time windows are repeatedly propagated and tightened to prune infeasible solutions and reduce the search space (Baptiste et al., 2001). In our approach, we calculate arc-based time windows for each trip, comprising earliest start, latest start, earliest finish, and latest finish. These time windows fulfill the same role of "locking in" partial schedules by revealing infeasible overlaps and ensuring resource-feasible allocations.

1.4. Roadmap

The remainder of this paper is structured as follows. Section 2 outlines the problem setting for the staggered routing problem. Section 3 details our matheuristic developed for solving the staggered routing problem. In Section 4, we provide an overview of our experimental design, while in Section 5, we present the results obtained from our numerical study. Finally, we conclude in Section 6 with some remarks and future research directions.

2. Problem statement

We begin by considering an offline problem wherein an AMoD operator coordinates an autonomous vehicle fleet to serve a given set of trips, e.g., trips arising within a day or a specific time window within a city. Each trip is operated by a single vehicle and can correspond to both on-duty, i.e., customer delivery, or off-duty, i.e., rebalancing activities, of the vehicle. Importantly, vehicles operate trips along a *prespecified* route, e.g., the shortest, within the road network. Within

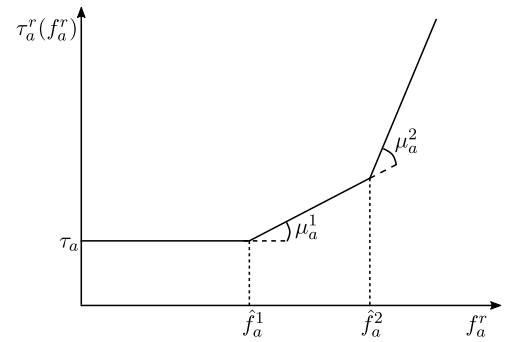


Fig. 1. Travel time of trip r on arc a as a function of the number of AMoD trips f_a^r.

this context, we focus on enhancing operational efficiency *after* vehicle routing decisions are taken. That is, we assume that the origin, destination, and route for each trip (equivalently, vehicle) are fixed. To do so, we aim to stagger trip departures, i.e., postpone trip departure times beyond their originally requested times, to reduce congestion caused by route overflows and thus minimize the fleet's travel time.

We model the road network within which the fleet operates as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$. The nodes $\mathcal{V}$ represent trip origins, destinations, and road intersections, while the arcs $\mathcal{A}$ are the road segments connecting these nodes. Given a set of trips $\mathcal{R}$, let a quadruple $(p^r, e^r, \ell^r, \bar{\sigma}^r)$ define a trip $r \in \mathcal{R}$, with p^r denoting the sequence of arcs composing the trip's fixed route through the network, e^r being the earliest departure from the trip's origin, ℓ^r being the latest acceptable arrival at the destination, and $\bar{\sigma}^r$ being the maximum staggering time applicable to trip departure.

A trip r on arc a incurs a travel time $\tau_a^r(f_a^r)$ that is the sum of a free-flow travel time τ_a and a congestion-induced delay, which we determine by the number of AMoD trips f_a^r on arc a when trip r starts traversing the arc.[1] We model such a congestion-induced delay as a piece-wise linear, non-decreasing, and convex function, resulting in a travel time over each arc such as that represented in Fig. 1. Without loss of generality, the total travel time encountered by trip r on arc a can thus be written as

$$\tau_a^r(f_a^r) = \tau_a + \max_{k \in K} \left\{ 0, \sum_{1 \le k' \le k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'}) \right\}, \tag{2.1}$$

where $\tau_a > 0$, $\mu_a^k > 0$ and $\hat{f}_a^k > 0$.

Remark 1. Two comments are in order. First, while other congestion models exist, e.g., simpler capacity models (Estandia et al., 2021) or more accurate microscopic models (Levin et al., 2017), we believe that our approach to modeling congestion provides a good balance between accuracy and computational complexity that is appropriate for the mesoscopic nature of our study. In this respect, it is important to remark that, while in mesoscopic studies, the relation between travel time and congestion is often taken to be a fourth-order polynomial (see, e.g., US Bur. Public Roads, 1964), our approach not only approximates such a function in a computationally convenient way, but it also allows to accommodate other, potentially different, relations. Second, when detecting the congestion level for a vehicle entering the arc, we ignore whether the other vehicles have just begun traversing the arc or are nearing completion. One possible way to mitigate this overestimation is to subdivide an arc into multiple segments, allowing for more accurate trip interactions. While this reduces aggregation errors, it also increases the computational burden, necessitating a balance between accuracy and tractability. We believe that the resulting trade-off between slightly

[1] Note that our formulation easily allows to consider the background presence of non-AMoD traffic, see Section 4 for details.

overestimating congestion and deriving a tractable model is reasonable for the scope of our study.

In this setting, a solution π is a vector with a departure time for each trip r. As we assume no vehicle idles once its trip has started, this information suffices to determine both the time as well as the number of vehicles f_a^r encountered by every vehicle when entering arc a of its route p^r. In turn, this sequence determines the travel times over all arcs and, thus, the trips' arrival times.

We require a solution π to satisfy the following constraints, which we collect in the feasible set Π:

(i) The departure time of each trip r must occur after e^r.
(ii) The departure time of each trip r can only be staggered for a maximum amount of time $\bar{\sigma}^r$.
(iii) The departures in π must ensure that the arrival at the destination of each trip r occurs before ℓ^r.

In this work, we seek a solution $\pi^\star \in \Pi$ that minimizes the total fleet travel time, i.e., the sum of the travel times each trip incurs over all arcs of its path

$$\min_{\pi\in\Pi} \mathrm{Z}(\pi) = \sum_{r\in\mathcal{R}} \sum_{a\in p^r} \tau_a^r(\pi), \tag{2.2}$$

where, with slight abuse of notation, we explicitly denote the fact that the travel time τ_a^r encountered on arc a by trip r depends on the entire vector of departure times.

Finally, while (2.2) describes the offline version of the problem we will consider, one can easily transform this offline setting into an online problem by discretizing time and taking decisions on batches of trips that arrive in the system according to a point process. In our numerical studies, we will comment on the difference between the maximum potential of staggering identified by solving the offline problem and the improvement attained by solving its online counterpart.

Remark 2. The dynamic traffic assignment literature offers various congestion models, such as the cell-transmission model, which captures complex phenomena like spillbacks (Adacher & Tiriolo, 2018). However, these models become computationally intractable for large-scale problems (Ziliaskopoulos et al., 2004). To ensure tractability and enable a MILP formulation (see Section 3.1), we use a piecewise linear delay function, a widely used approximation of nonlinear volume-delay functions in static traffic assignment (see, e.g., Bang & Malikopoulos, 2022). Note that our congestion model is akin to Vickrey's bottleneck model (Vickrey, 1969) in its discretized version (Otsubo & Rapoport, 2008), where n vehicles attempt to traverse an arc at distinct departure times t, with the traversal being characterized by a free-flow travel time and a capacity threshold. Once this threshold is surpassed, a bottleneck arises, modeled as a queue forms at the arc's entrance, regulated by a first-in-first-out (FIFO) policy, with vehicles leaving the queue as others complete their traversal of the bottleneck. In our model, the travel time of an arc, as defined in (2.1), consists of a free flow travel time τ_a and a delay akin to the waiting time in a vertical queue. The waiting time increases when the vehicle count on the link exceeds $\hat{f}_a$. Unlike in Vickrey's model, the waiting time in our model depends on the number of vehicles concurrently on the arc and in the queue at t, and is calculated based on a function similar to that shown in Fig. 1. It is worth mentioning that as a technical difference, our model does not enforce FIFO, as it allows shorter waiting times for later departures. Accordingly, it remains an approximation of the queuing dynamic in Vickrey's model that may show local deviations for single vehicles but sums up to a comparable total travel time.

3. Methodology

The staggered routing problem as introduced in Section 2 is NP-hard, which we show in Appendix A. This motivates us to develop an efficient matheuristic that allows to solve large-scale instances in the following. First, we introduce a MILP that allows us to obtain solutions for Problem (2.2) in Section 3.1. Towards this goal, we introduce a basic model that uses an indicator function to quantify the aggregated flow, before we elaborate on how to model the respective indicator constraints. Second, we show how to tighten the formulation on the MILP by leveraging the problem structure in Section 3.2. Third, we present a matheuristic that combines our MILP with a local search scheme to solve large real-world instances in Section 3.3. Finally, Section 3.4 adapts the previously developed algorithm to solve the online counterpart of our problem in a rolling-horizon fashion.

3.1. Mixed integer linear program

We aim to formulate (2.2) as a MILP. Towards this goal, note that minimizing the total travel time is equivalent to minimizing the congestion-induced delay, as we have no control over the arcs' free-flow travel times τ_a. The objective therefore reads

$$\min \sum_{r\in\mathcal{R}} \sum_{a\in p^r} \max_{k\in K} \left\{ 0, \sum_{1\le k'\le k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'}) \right\}.$$

With this in mind, we obtain a linear representation of this non-linear objective using an epigraphic reformulation (Boyd & Vandenberghe, 2004). Specifically, we introduce delay variables x_a^r and enforce the constraint $x_a^r \ge \sum_{k'=1}^{k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'})$ for all arcs, trips, and $k \in K$. To represent the entry time of trip r on arc a on its path, we introduce the continuous variable s_a^r. Further, we let $\delta^r(a)$ denote the successor arc of arc a on route p^r, and $\underline{a}^r$, $\bar{a}^r$ denote the first and last arcs of trip r, respectively. Finally, we let $\mathcal{R}_a$ denote the set of all trips that transit through arc a. Then, problem (2.2) can be written as:

$$\min \sum_{r\in\mathcal{R}} \sum_{a\in p^r} x_a^r, \tag{3.1a}$$

s.t.

$$x_a^r \ge \textstyle\sum_{k'=1}^{k} \mu_a^{k'} \cdot \left(f_a^r - \hat{f}_a^{k'}\right), \qquad \forall k \in K, r \in \mathcal{R}, a \in p^r \tag{3.1b}$$

$$s_{\delta^r(a)}^r = s_a^r + \tau_a + x_a^r, \qquad \forall r \in \mathcal{R}, a \in p^r \setminus \bar{a}^r \tag{3.1c}$$

$$f_a^r = \textstyle\sum_{r'\in\mathcal{R}_a\setminus\{r\}} \mathbb{I}(s_a^{r'} \le s_a^r < s_a^{r'} + \tau_a + x_a^{r'}), \qquad \forall r \in \mathcal{R}, a \in p^r \tag{3.1d}$$

$$e^r \le s_{\underline{a}^r}^r \le e^r + \bar{\sigma}^r, \qquad \forall r \in \mathcal{R} \tag{3.1e}$$

$$s_{\bar{a}^r}^r + \tau_{\bar{a}^r} + x_{\bar{a}^r}^r \le \ell^r, \qquad \forall r \in \mathcal{R} \tag{3.1f}$$

$$x_a^r, s_a^r, f_a^r \ge 0. \qquad \forall r \in \mathcal{R}, a \in p^r \tag{3.1g}$$

Our objective (3.1a) jointly with Constraints (3.1b) minimizes the total delay over all trips and arcs. Constraints (3.1c) propagate forward the time at which trip r enters arc a along its route p^r, while Constraints (3.1d) calculate the number of vehicles encountered by each trip r on arc a. This is achieved using an indicator function $\mathbb{I}(\cdot)$, which is set to one if trip r enters arc a when r' is traversing the same arc. Constraints (3.1e)–(3.1f) ensure feasible start and drop-off times. Constraints (3.1g) state the variable domains.

To convert the latter model into a MILP, we replace the indicator function in Constraints (3.1d) with big-M constraints that replicate its logic. To do so, we introduce binary variables $\alpha_a^{r,r'}, \beta_a^{r,r'}$ and $\gamma_a^{r,r'}$ for each pair of distinct trips (r, r') traversing the same arc, whose logic is as follows:

$$\alpha_a^{r,r'} = \begin{cases} 1, \text{ if } s_a^r \ge s_a^{r'} \\ 0, \text{ otherwise} \end{cases} \qquad \beta_a^{r,r'} = \begin{cases} 1, \text{ if } s_a^r < s_a^{r'} + \tau_a + x_a^{r'} \\ 0, \text{ otherwise} \end{cases}$$

$$\gamma_a^{r,r'} = \begin{cases} 1, \text{ if } \alpha_a^{r,r'} + \beta_a^{r,r'} = 2 \\ 0, \text{ otherwise} \end{cases}$$

We enforce this logic and therefore evaluate f_a^r by replacing constraints (3.1d) with the following:

$$s_a^r - s_a^{r'} + \varepsilon \le M_1 \cdot \alpha_a^{r,r'}, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.1h}$$

$$s_a^{r'} - s_a^r \le M_2 \cdot (1 - \alpha_a^{r,r'}), \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.1i}$$

$$s_a^{r'} + \tau_a + x_a^{r'} - s_a^r \le M_3 \cdot \beta_a^{r,r'}, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.1j}$$

$$s_a^r - s_a^{r'} - \tau_a - x_a^{r'} + \varepsilon \le M_4 \cdot (1 - \beta_a^{r,r'}), \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.1k}$$

$$\gamma_a^{r,r'} - \alpha_a^{r,r'} - \beta_a^{r,r'} \ge -1, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.1l}$$

$$2\gamma_a^{r,r'} - \alpha_a^{r,r'} - \beta_a^{r,r'} \le 0, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r' \tag{3.1m}$$

$$f_a^r = \textstyle\sum_{r' \in \mathcal{R}_a \setminus \{r\}} \gamma_a^{r,r'} \qquad \forall a \in \mathcal{A}, r \in \mathcal{R}_a \tag{3.1n}$$

where ε is a small constant and M_i are sufficiently large numbers.[2] Constraints (3.1h) ensure that $\alpha_a^{r,r'}$ takes the value one if $s_a^r \ge s_a^{r'}$, while Constraints (3.1i) ensure that it equals zero otherwise. Similarly, Constraints (3.1j)–(3.1k) guarantee the correct behavior of $\beta_a^{r,r'}$. Constraints (3.1l) ensure that $\gamma_a^{r,r'}$ equals one when both $\alpha_a^{r,r'}$ and $\beta_a^{r,r'}$ are one, while Constraints (3.1m) enforce that it equals zero otherwise. Finally, Constraints (3.1n) computes f_a^r as the count of $\gamma_a^{r,r'}$ variables equal to one.

To minimize the M_i constants in the big-M constraints, we derive the following relationships from Constraints (3.1h)–(3.1k):

$$M_1 \ge \max(s_a^r - s_a^{r'} + \varepsilon) = \bar{e}_a^r - \underline{e}_a^{r'} + \varepsilon, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r'$$

$$M_2 \ge \max(s_a^{r'} - s_a^r) = \bar{e}_a^{r'} - \underline{e}_a^r, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r'$$

$$M_3 \ge \max(s_a^{r'} + \tau_a + x_a^{r'} - s_a^r) = \bar{\ell}_a^{r'} - \underline{e}_a^r, \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r'$$

$$M_4 \ge \max(s_a^r + \tau_a + x_a^r - s_a^{r'} + \varepsilon) = \bar{\ell}_a^r - \underline{e}_a^{r'} + \varepsilon. \qquad \forall a \in \mathcal{A}, r, r' \in \mathcal{R}_a, r \neq r'$$

Here, the variables $\underline{e}_a^r$, $\bar{e}_a^r$, and $\bar{\ell}_a^r$ define trip arc time windows, representing, respectively, the earliest departure time, latest departure time, and latest traversal completion time for trip r on arc a. In Section 3.2.1, we present an algorithm to compute tight arc time windows, which can be directly utilized to compute the big-M values.

3.2. Constraint reduction and tightening

Although the formulation in Section 3.1 is a MILP, it poses significant computational challenges due to the number of big-M constraints required to check if a given pair of vehicles travels simultaneously on the same arc. Specifically, each constraint in (3.1h)–(3.1m) needs to be repeated for all pairs of trips that transit through each given arc. This formulation results, unfortunately, in prohibitive solution times, even for modest problem sizes, as the continuous relaxation used in the solution of the MILP performs poorly. To enhance computational efficiency, we leverage the problem structure to identify and remove as many redundant big-M constraints as possible in the following.

A key observation is that not every distinct pair of trips traversing the same arc may influence each other's travel time, largely because each trip is constrained by specific time windows. However, the original problem formulation introduced in Section 2 only defines these windows for each entire trip, not for individual arcs. Based on these trip-specific time windows, our approach constructs arc-specific time windows to identify trips that, by traversing the arc simultaneously, may incur a delay. We then leverage these bounds to build a representation of the problem on a multidigraph, in which parallel arcs represent identical road segments, but each arc is associated with either a set of potentially conflicting trips, i.e., trips that may influence each other's travel time and thus contain conflicting arcs, i.e., arcs on which trip induced congestion may arise, or trips that will not delay each other, containing only non-conflicting arcs. This differentiation allows us to apply big-M constraints selectively, focusing only on conflicting arcs and relevant trip pairs, thereby significantly reducing the number of big-M constraints appearing in the MILP. Additionally, we explore methods to merge or exclude non-conflicting arcs from the model, further minimizing its complexity and the computational overhead of the local search method discussed later in the section.

In the rest of this section, we will proceed as follows. In Section 3.2.1, we detail how to determine arc-specific time windows for each trip. In Sections 3.2.2 and 3.2.3, we detail how we derive the multigraph representation introduced above. Building on this, we ultimately reduce the number of big-M constraints required in our MILP model.

3.2.1. Determining arc-dependent time windows

To distinguish trips that may conflict from trips that do not, we construct upper and lower bounds on the entry and exit times for each given arc on a trip. At a high level, the goal is to make these bounds as tight as possible while ensuring that every feasible solution satisfies them, which allows to reduce the number of big-M constraints.

Specifically, we let $\underline{e}_a^r$, $\bar{e}_a^r$ represent the earliest and latest *entry* time of trip r in arc a. Similarly, we use $\underline{\ell}_a^r$, $\bar{\ell}_a^r$ to represent earliest and latest *exit* times. Note, however, that knowledge of the earliest entry time $\underline{e}_a^r$ and latest exit time $\bar{\ell}_a^r$ for all arcs on a trip suffices to reconstruct also the other quantities, i.e., the latest entry and the earliest exit over all the arcs on that trip. In fact, by continuity, the latest entry on an arc $\bar{e}_a^r$ must equal the latest exit from the previous arc on that trip, while the earliest exit from an arc $\underline{\ell}_a^r$ must equal the earliest entry on the subsequent arc on that trip. This is the case for all arcs in the trip except for the very first and last arc. For the first arc, we set the latest entry as $\bar{e}_{\underline{a}^r}^r = e^r + \bar{\sigma}^r$, while for the last arc we determine the earliest exit by letting the vehicle travel in free flow through that arc so that $\underline{\ell}_{\bar{a}^r}^r = \underline{e}_{\bar{a}^r}^r + \tau_a$.

Thus, in the following, we focus on determining only the earliest entry $\underline{e}_a^r$ and the latest exit $\bar{\ell}_a^r$ times. The idea is to initialize these quantities by propagating the earliest entry times over subsequent arcs on a trip as if vehicles were traveling in free-flow, starting from the earliest departure e^r at the first arc. Similarly, for the latest exit times, which we propagate backward over preceding arcs on a trip, starting from the latest arrival at the last arc ℓ^r. In a second phase, we then refine and tighten these time windows by bounding the minimum and maximum congestion encountered by each trip over its arcs. We now describe both of these steps in further detail and refer to Algorithm 1 for a formal overview of the approach.

Algorithm 1 Compute arc-specific time windows

Input: Set of trips $\mathcal{R}$

```
Q, e, ℓ̄, x ← INITIALIZE(R)
while Q is not empty do
    (a, r) ← Q.POP()                    ▷ Extract tuple with highest priority.
    e, x ← UPDATEEARLIESTENTRYTIMES(a, r)
    ℓ̄ ← UPDATELATESTEXITTIMES(a, r)
LB ← GETINITIALLOWERBOUND(x)
return (e, ℓ̄, LB)
```

Initialization. For each trip, we set the earliest entry time $\underline{e}_a^r$ for its first arc equal to the earliest departure time of the trip. We then calculate the earliest entry times for the subsequent arc by adding the nominal travel time of the preceding arc, that is $\underline{e}_{\delta^r(a)}^r = \underline{e}_a^r + \tau_a$ for all $a \in p^r \setminus \bar{a}^r$. For each trip, we set the latest exit time $\bar{\ell}_a^r$ for its last arc equal to the latest arrival time of the trip ℓ^r. We then calculate the

[2] Note that the use of a small constant ε is necessary. Indeed, in its absence, given a pair of distinct trips (r, r') jointly traversing arc a, for s_a^r being equal to $s_a^{r'}$, setting $\alpha_a^{r,r'}$ to either zero or one would satisfy Constraints (3.1h)–(3.1i).

latest exit time for the preceding arc by subtracting the nominal travel time of the arc, that is $\bar{\ell}^r_a = \bar{\ell}^r_{\delta^r(a)} - \tau_{\delta^r(a)}$ for all $a \in p^r \setminus \bar{a}^r$.

We then construct a queue whose elements are the tuples indexed by (a, r), i.e., arc and trip, and sort such queue based on the smallest earliest entry times. We then process, in order, one element of the queue at a time to update its earliest entry and latest exit time. First, we describe the method for updating time windows. Following that, we detail how employing a priority queue results in tighter bounds compared to an unsorted approach.

Updating earliest entry times. Given an element (a, r), we now update, i.e., move forward in time, the earliest entry times on arcs subsequent to a to tighten the time windows. Towards this goal, we compute the minimum conflicts that trips r inevitably incur on arc a and denote it with $\underline{f}^r_a$. This quantity can be determined by leveraging the existing time windows as

$$\underline{f}^r_a = \sum_{r' \in \mathcal{R}_a \setminus \{r\}} \mathbb{I}(\underline{e}^r_a > \bar{e}^{r'}_a \wedge \bar{e}^r_a < \underline{\ell}^{r'}_a).$$

We denote $\underline{x}^r_a$ as the delay that r incurs on arc a in presence of $\underline{f}^r_a$. As the earliest entries on arcs subsequent to a are surely delayed by $\underline{x}^r_a$, we update $\underline{e}^r_{a'} \leftarrow \underline{e}^r_{a'} + \underline{x}^r_a$ for every arc $a' \in p^r$ such that $\underline{e}^r_{a'} > \underline{e}^r_a$.

Updating latest exit times. To update the latest exit time $\bar{\ell}^r_a$, we determine the maximum number of conflicts, $\bar{f}^r_a$, that a trip r can encounter on arc a. A simple counting argument reveals that this is equal to the number of other trips $r' \in \mathcal{R}_a$ whose time windows $[\underline{e}^{r'}_a, \bar{\ell}^{r'}_a]$ intersect $[\underline{e}^r_a, \bar{e}^r_a]$. We then update the latest exit time from arc a based on the latest entry in such arc and the maximum number of conflicts computed, that is, $\bar{\ell}^r_a \leftarrow \min(\bar{\ell}^r_a, \bar{e}^r_a + \tau^r_a(\bar{f}^r_a))$.

Computing an initial lower bound. We conclude Algorithm 1 by calculating an initial lower bound LB, which already allows evaluating the quality of a solution π. After finalizing the computation of arc-dependent time windows, the initial lower bound results from summing up the minimum delays that trips unavoidably incur on each arc:

$$\mathrm{LB} = \sum_{r \in \mathcal{R}} \sum_{a \in p^r} \underline{x}^r_a.$$

The priority queue. The priority queue, organized by ascending earliest entry times $\underline{e}^r_a$, improves upon an unsorted approach to processing the sequence of the time window for two main reasons:

(i) Processing pair (a, r) enables us to tighten both the earliest entry times for arcs after a in r's route, as well as the latest exit time from a. In this context, proceeding in an ascending order aligns with this forward-looking update mechanism.

(ii) Ordering by $\underline{e}^r_a$ allows to tighten the maximum number of conflicts $\bar{f}^r_a$. As seen above, r may conflict with r' on a if $[\underline{e}^r_a, \bar{e}^r_a]$ overlaps with $[\underline{e}^{r'}_a, \bar{\ell}^{r'}_a]$. At the time the algorithm processes pair (a, r), the forward-looking update logic highlighted in point i) ensures $\underline{e}^r_a$, $\underline{e}^{r'}_a$ and $\bar{e}^r_a$ to be as tight as possible. Nevertheless, $\bar{\ell}^{r'}_a$ may or may not be as tight as possible, depending on the following case distinction:

(1) If $\underline{e}^r_a > \underline{e}^{r'}_a$, the algorithm processed r' before r on a. Thus, $\bar{\ell}^{r'}_a$ is as tight as possible consequent to the update logic of point i).

(2) If $\underline{e}^r_a < \underline{e}^{r'}_a$, r is processed before r'. Although $\bar{\ell}^{r'}_a$ is not as tight as possible, the determinant for overlap is $\underline{e}^{r'}_a$, which is as tight as possible.

Note that changes to a trip's earliest entry time can disrupt the queue's ordering. To address this, we maintain the queue's priorities to ensure that tuples are processed in an ascending time sequence.

Discussion. Notice that our current method tends to overestimate the worst-case delay by focusing solely on the bounds related to the specific arc under consideration. We do not dynamically adjust these bounds based on changing scenarios across consecutive arcs. In Appendix C, we provide a small numerical example demonstrating how such a conservative approach might not capture the interplay of delays across arcs and how one could derive tighter bounds. As the scalability of the approach largely hinges on the tightness of departure bounds, addressing this limitation through a dynamic bound adaptation process presents a promising direction for future research.

Moreover, one can iterate the bounding procedure multiple times to achieve more precise bounds. This can be achieved by iteratively populating Q with newly computed earliest departure bounds and running the procedure until convergence.

3.2.2. Calculating conflicting sets

After determining arc-specific time windows for all trips, we proceed by calculating conflicting sets $\mathcal{S}^i_a$, i.e., sets containing pairs of trips that can incur delay by conflicting. This process partitions the set of trips using a given arc over time into smaller subsets, where each subset groups trips that share a specific time window. Notably, trips in one subset do not overlap with those in another in their utilization of the arc. To achieve this, we define for each arc a a set $\mathcal{P}_a$ that contains all pairs of distinct trips belonging to $\mathcal{R}_a$ that might potentially conflict:

$$\mathcal{P}_a := \{(r, r') \in \mathcal{R}_a \times \mathcal{R}_a \mid r \neq r' \wedge [\underline{e}^r_a, \bar{e}^r_a] \cap [\underline{e}^{r'}_a, \bar{\ell}^{r'}_a] \neq \emptyset\}.$$

Note that we use tuples because potential conflicts are asymmetric. Specifically, in our definition, a trip r may conflict with another trip r', but this does not imply the reverse. By construction, $\mathcal{P}_a$ contains all tuples representing potential conflicts between trips that utilize a. Still, within $\mathcal{P}_a$, distinct subsets of tuples built on trips that do not appear in any other subset may emerge. This separation enables us to identify subsets of trips that may conflict with each other but not with other trips utilizing arc a. To isolate these dependencies, we split $\mathcal{P}_a$ into subsets of maximal size such that each subset contains only tuples formed from trips exclusive to that particular subset, ensuring that no trip is included in more than one subset. Technically, we partition $\mathcal{P}_a$ into n sets $\mathcal{P}^i_a$ of maximal size that satisfy the following conditions:

(i) The subsets $\mathcal{P}^i_a$ are pairwise disjoint and their union constitutes the entire set $\mathcal{P}_a$, i.e., $\mathcal{P}_a := \bigcup_{i=1}^n \mathcal{P}^i_a$ and $\mathcal{P}^i_a \cap \mathcal{P}^j_a = \emptyset$ for all $i \neq j$.

(ii) For any subset $\mathcal{P}^i_a$ containing more than one pair, it holds that for each pair $(r, r') \in \mathcal{P}^i_a$, there exists another pair $(r'', r''') \in \mathcal{P}^i_a$ such that the intersection of their components is non-empty: $\{r, r'\} \cap \{r'', r'''\} \neq \emptyset$. If no such pair (r'', r''') exists, then (r, r') constitutes a singleton subset in the partition.

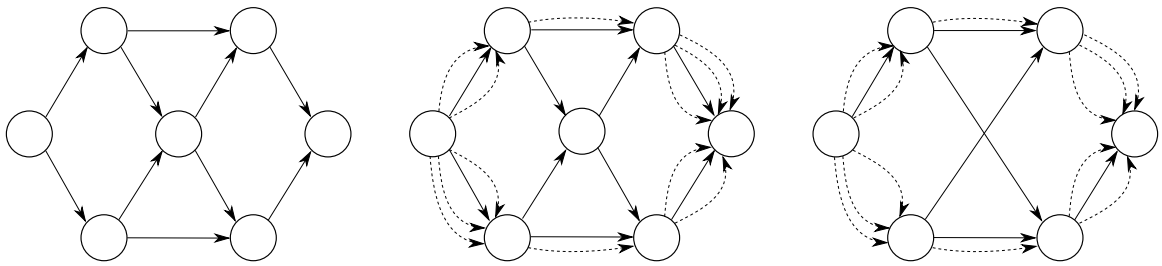
With $\mathcal{P}^i_a$ containing all tuples relative to a subset of trips, we can finally identify those trips that may incur a delay. According to Eq. (2.1), this is the case when the number of conflicts that a trip in $\mathcal{P}^i_a$ has, i.e., the number of trips traversing an arc at the same time, exceeds the first threshold $\hat{f}^1_a$, beyond which it is not possible to travel on arc a at free-flow speed. Then, we can tighten the maximum number of conflicts $\bar{f}^r_a$ by counting the number of pairs in $\mathcal{P}^i_a$ in which a trip r appears as the first element. Accordingly, we filter our partition of $\mathcal{P}^i_a$ to derive conflicting sets $\mathcal{S}^i_a$, each containing trips that may induce delay, as:

$$\mathcal{S}^i_a := \left\{(r, r') \in \mathcal{P}^i_a \mid \bar{f}^r_a \geq \hat{f}^1_a\right\}.$$

Note that we omit r' in the set definition, as our goal is identifying trips r with sufficient conflicts (r, r') in $\mathcal{P}^i_a$ to meet the free flow threshold $\hat{f}^1_a$.

3.2.3. Multigraph representation and MILP constraints sparsening

After identifying all conflicting sets, we aim to modify our original graph $\mathcal{G}$ towards a multigraph representation. This allows us to maintain Constraints (3.1h)–(3.1n) at a minimum. Fig. 2 shows an example of our graph modification that consists of three steps: in the first step, we transition to a multigraph by adding a parallel arc for each conflicting set; in the second step, we sparsen our graph by removing arcs that are no longer used and merging sequences of conflict-free arcs; in the final step, we redefine the conflict-checking constraint sets to include only relevant trip pairs.



(a) Original graph $\mathcal{G}$ (b) $\mathcal{G}$ expanded by conflicting arcs (c) Non-conflicting arcs processing

Fig. 2. Schematic representation of the graph transformation process upon identification of conflicting sets: (a) initial graph structure. (b) expansion with conflicting arcs (dashed lines). (c) result of merging sequences of non-conflicting arcs and removing unused arcs. The merging process involves removing the central node and replacing its incident arcs with non-conflicting arcs with the original arcs' cumulative length.

Step 1: For each conflicting set $\mathcal{S}_a^i$, we add a conflicting arc $\hat{a}_i$ as a parallel arc to its original counterpart a. This arc shares the characteristics of a, i.e., it has the same start point, endpoint, nominal travel time, and capacity. After adding a conflicting arc $\hat{a}_i$, we modify the routes that belong to the trips contained in $\mathcal{S}_a^i$ by replacing the original arc a with the respective conflicting arc $\hat{a}_i$, such that all trips that share a potential conflict traverse the same conflicting arc. Let the set of conflicting arcs be $\hat{\mathcal{A}} \subset \mathcal{A}$. After Step 1, only trips that cannot induce delay on an arc a will still use arc $a \in \mathcal{A} \setminus \hat{\mathcal{A}}$. Accordingly, we will refer to trips that utilize only arcs $a \in \mathcal{A} \setminus \hat{\mathcal{A}}$ as non-conflicting trips and to the respective arcs as non-conflicting arcs. We remove trips from $\mathcal{R}$ that traverse only non-conflicting arcs.

Step 2: To sparsen our arc set, we aim to merge as many non-conflicting arcs as possible. We combine sequences of non-conflicting arcs in trip routes into single arcs, which we add to $\mathcal{A}$. Each new arc starts at the origin of the first arc in the sequence and ends at the destination of the last arc, and its length equals the sum of the lengths of all arcs in the sequence. We update accordingly the routes p^r in which such sequences appear. Finally, we remove arcs from $\mathcal{A}$ that do not appear in any trip's route and update $\mathcal{R}_a$ for every arc in $\mathcal{A}$ accordingly. In Section 4, we illustrate how this procedure impacts the sizes of sets $\mathcal{R}_a$ and the number of arcs in the multigraph representation in our experiments.

Step 3: The remaining task involves updating the definition of Constraints (3.1h)–(3.1m) as follows:

Constraints (3.1h)–(3.1m), $\forall a \in \hat{\mathcal{A}}, (r, r') \in \mathcal{P}_a$

thereby applying these constraints exclusively to conflicting arcs, $\hat{\mathcal{A}}$, and to each pair of trips that might conflict and potentially cause delays. Finally, we accordingly tighten the corresponding M_i values by incorporating the refined time window bounds into the constraints derived in Section 3.1.

3.3. Matheuristic

We develop a matheuristic incorporating our MILP into a local search scheme to find good solutions for large real-world instances. While this matheuristic is not guaranteed to find an optimal solution, it leverages the MILP from Section 3.1 and can thus detect an optimal solution if encountered. Algorithm 2 shows a high-level pseudo-code of our matheuristic. After creating an initial solution π that is agnostic to staggering, we apply a local search to improve it by staggering its trip departures. We then aim to further improve π by iterating between our MILP and our local search: whenever we find a new incumbent by solving the MILP, we stop and improve this incumbent with our local search. Once the local search terminates, we feed its final solution back to warmstart the MILP. Our matheuristic stops either when the MILP certifies optimality of π, i.e., $Z(\pi) = \text{LB}$, or after a maximum time η elapsed. Note that even if the MILP cannot certify optimality within the computational time limit, our matheuristic has the advantage of providing a lower bound to indicate the quality of π. In the remainder of this section, we detail our construction routine to obtain an initial solution (Section 3.3.1) before we describe our local search (Section 3.3.2).

Algorithm 2 Matheuristic

Input earliest departure times $\boldsymbol{e}$, initial lower bound LB, time limit η
$\pi \leftarrow$ constructSolution($\boldsymbol{e}$)
$\pi \leftarrow$ localSearch(π)
while time elapsed $\leq \eta$ **do**
  LB, $\pi \leftarrow$ solveMILP(LB, π)
  if $Z(\pi) = \text{LB}$ **then**
    return π
  $\pi \leftarrow$ localSearch(π)
return π

3.3.1. Constructing initial solutions

To construct an initial solution π, we set its departure time variables s_a^r, delay variables x_a^r and conflict counting variables f_a^r to infinity. We then sort the trip-specific earliest departures e^r of every trip r into a priority queue Q and initialize an empty set $\mathcal{T}_a$ for each arc a to track when each trip completes the traversal of a. Then, we construct an initial solution iteratively:

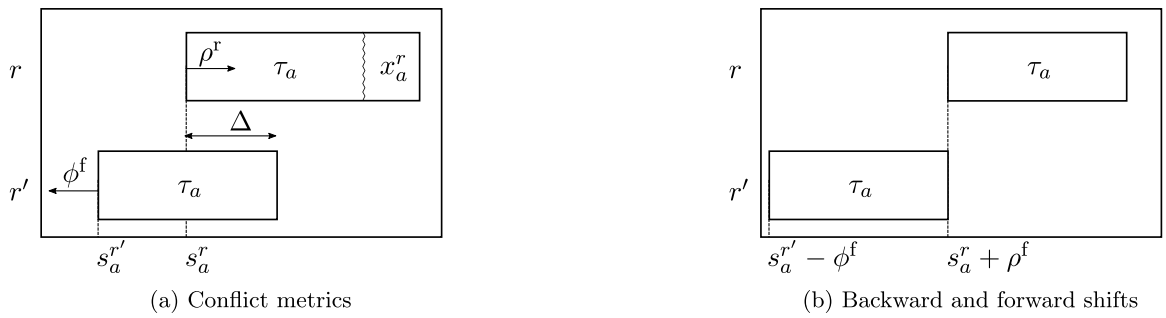
(i) We start by extracting the departure time s_a^r with the highest priority from Q.
(ii) We then set $f_a^r = |\{t \in \mathcal{T}_a \mid s_a^r < t\}|$, that is the number of trips that have not completed their traversal of arc a by time s_a^r, and update $x_a^r = \max_{k \in K}\{0, \sum_{k'=1}^{k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'})\}$.
(iii) Finally, we add the time in which the arc traversal is completed, which is $s_a^r + x_a^r + \tau_a$, to $\mathcal{T}_a$ by setting $\mathcal{T}_a = \mathcal{T}_a \cup (s_a^r + x_a^r + \tau_a)$. If a is not the last arc of route p^r, we insert $s_a^r + x_a^r + \tau_a$ into Q.

We repeat these steps until Q is empty. Note that once we obtained a feasible solution, it is possible to compute values for the MILP's binary variables $\alpha_a^{r,r'}, \beta_a^{r,r'}$ and $\gamma_a^{r,r'}$ with a support function checking their activation conditions as introduced in Section 3.1.

3.3.2. Local search

Our local search aims to iteratively remove conflicts based on a priority queue, aiming to first resolve conflicts that induce the largest delays. Let the tuple (r, r', a) index the conflict that r has with r' on arc a. Given a conflict (r, r', a), we resolve it by either shifting r backward or r' forward in time without violating the trips' time windows. In this context, we compute additional quantities for each conflict, utilizing the relations shown in Fig. 3. Specifically, consider a conflict (r, r', a) with $s_a^r > s_a^{r'}$. We then compute the time overlap Δ between r and r', which denotes how much staggering we need to apply to resolve the conflict. Formally:

$$\Delta = s_a^{r'} + \tau_a + x_a^{r'} - s_a^r + \varepsilon.$$



(a) Conflict metrics (b) Backward and forward shifts

Fig. 3. Figure (a) illustrates the quantities computed in the local search for resolving a conflict (r, r', a), where r incurs delay x_a^r. It illustrates the time overlap Δ, the feasible backward shift ρ^{f}, and forward shift ϕ^{f}. Figure (b) depicts trips r and r' after applying these shifts, resulting in conflict resolution and the consequent elimination of the arc delay.

Clearly, $\Delta = \phi = \rho$ indicates the required forward shift ϕ for r' and the required backward shift ρ for r to resolve the conflict by staggering just one of the two trips. However, shifting a trip by Δ may cause time window violations. Therefore, we limit ρ and ϕ to their feasible counterparts, ρ^{f} and ϕ^{f}. These values represent the maximum backward and forward time shifts applicable without violating the time window of each trip. Formally, ϕ^{f} reads:

$$\phi^{\mathrm{f}} = \min(s^{r'} - e^{r'}, \Delta),$$

while ρ^{f} reads:

$$\rho^{\mathrm{f}} = \min(\bar{\sigma}^r - (s^r - e^r), \Delta),$$

with s^r denoting the starting time of a trip r. Based on these quantities, Algorithm 3 provides the pseudocode for our local search: given an initial solution π, we sort conflicts into a priority queue Y, thus ordering conflicts based on their induced delay in decreasing order (l. 1). We then process Y until it is empty (l. 2). In each iteration, we pop the first conflict from Y (l. 3) and compute ϕ^{f}, ρ^{f}, and Δ as defined above (l. 4). Only if the sum of ϕ^{f} and ρ^{f} allows to compensate Δ (l. 5), we resolve the conflict (l. 6). If this leads to an improved solution (l. 7), we update our solution (l. 8) as well as our priority queue (l. 9) since new conflicts may arise. If the conflict cannot be resolved, we directly proceed with the next item in Y without updating π or Y. In the remainder of this section, we elaborate on constructing and evaluating a neighborhood to create a new solution that resolves a conflict.

While our local search follows a rather simple greedy repair selection based on conflict severity, its complexity lies in efficiently evaluating a neighborhood, i.e., computing the impact of resolving one conflict on all other trips' departure times. In the best case, resolving a conflict induces changes only in arc entry times for the conflicting trips, while in the worst case, resolving a conflict can lead to a very large cascade of changes in other trip departures. Algorithm 4 shows how we efficiently update our solution when resolving a conflict: we first apply a minimum staggering to the departures of r and r'. When staggering a trip departure time to resolve a conflict, we insert a corresponding tuple (r, a), consisting of the trip's index r and its route's first arc a into a priority queue Q, which we use to maintain trips that require recalculating arc entry times due to the applied staggering. We sort the tuples in Q in ascending order based on their arc departure times s_a^r (l. 1). After resolving the initial conflict, we start processing trips in Q (l. 3) by checking whether the trip's updated departure time s_a^r creates a change in the arc entry times of another trip r' (l. 4). In this case, we insert r' into Q. If inserting r' into Q invalidates a further propagation of arc entry times for trip r – i.e., if $s_a^{r'} < s_a^r$, which disrupts the order of our priority queue – we reinsert s_a^r into Q and restart evaluating Q (l. 5&6). If s_a^r does not induce changes in other trips, we recursively propagate the change in trip r's arc entry times on subsequent arcs until a change arises (l. 7–l. 13). Our overall update ends once Q is empty and all arc entry times have been correctly adjusted. To conclude this section, we detail how we stagger departure times, check for solution changes,

Algorithm 3 Local Search

Input: solution π
1: $Y \leftarrow$ IDENTIFYCONFLICTS(π)
2: **while** Y is not empty **do**
3: $(r, r', a) \leftarrow Y$.POP()
4: $\phi^{\mathrm{f}}, \rho^{\mathrm{f}}, \Delta \leftarrow$ ANALYZECONFLICT(r, r', a)
5: **if** $\phi^{\mathrm{f}} + \rho^{\mathrm{f}} \geq \Delta$ **then**
6: $\bar{\pi} \leftarrow$ RESOLVECONFLICT($\pi, r, r', \phi^{\mathrm{f}}, \Delta$)
7: **if** $Z(\bar{\pi}) < Z(\pi)$ **then**
8: $\pi \leftarrow \bar{\pi}$
9: $Y \leftarrow$ IDENTIFYCONFLICTS($\bar{\pi}$)
10: **return** π

Algorithm 4 RESOLVECONFLICT

Input: solution π, trips r and r', feasible shifts ϕ^{f} and ρ^{f}, time overlap Δ
1: Q $\leftarrow$ STAGGERDEPARTURES($r, r', \phi^{\mathrm{f}}, \rho^{\mathrm{f}}, \Delta$)
2: **while** Q is not empty **do**
3: $(r, a) \leftarrow$ Q.POP()
4: $r', x_a^r \leftarrow$ UPDATESOLUTION(r, a)
5: **if** $r' \neq$ FALSE **then**
6: Q $\leftarrow$ INSERTTRIPS(r, r', a)
7: **else**
8: **while** $r' =$ FALSE **do**
9: $s_{\delta^r(a)}^r \leftarrow s_a^r + \tau_a + x_a^r$
10: $a \leftarrow \delta^r(a)$
11: $r', x_a^r \leftarrow$ UPDATESOLUTION(r, a)
12: **if** $r' \neq$ FALSE **then**
13: Q $\leftarrow$ INSERTTRIPS(r, r', a)
14: **return** π

and maintain the priority queue. STAGGERDEPARTURES$(r, r', \phi^{\mathrm{f}}, \rho^{\mathrm{f}}, \Delta)$: To resolve a conflict, we need to shift either r or r' or both in time by Δ. Clearly, it is desirable to shift only one of these trips in time to keep the number of newly arising conflicts and entry time changes at a minimum. Accordingly, we proceed as follows to resolve a conflict:

1. If $\rho^{\mathrm{f}} = \Delta$ we adjust $s^r \leftarrow s^r + \Delta$ and push $(r, \underline{a}^r)$ into Q.
2. If a conflict cannot be resolved by shifting only trip r in time, i.e., $\rho^{\mathrm{f}} < \Delta$, we stagger both trips such that $s^r \leftarrow s^r + \rho^{\mathrm{f}}$ and $s^{r'} \leftarrow s^{r'} - \phi^{\mathrm{f}}$ and push $(r, \underline{a}^r)$ as well as $(r', \underline{a}^{r'})$ into Q.

UPDATESOLUTION(r, a): After staggering trips to resolve a conflict, we need to update the arc entry times of these trips as well as the arc entry times of all other trips that might be affected by the change. To do so, we first compute the number of conflicts f_a^r that arise for the respective trip and arc as:

$$f_a^r = \sum_{r' \in \mathcal{R}_a \setminus \{r\}} \mathbb{I}(s_a^{r'} \leq s_a^r < s_a^{r'} + \tau_a + x_a^{r'}),$$

which subsequently allows us to update the corresponding delay as:

$$x_a^r = \max_{k \in K} \left\{ 0, \sum_{1 \le k' \le k} \mu_a^{k'} \cdot (f_a^r - \hat{f}_a^{k'}) \right\}.$$

We can then check whether a new conflict arises or an existing conflict gets resolved between r and some other trip r' due to the updated entry time of r. If either is the case, we return the index of trip r'; otherwise, we return a FALSE flag to indicate that no new conflict was induced or resolved by staggering r.

INSERTTRIPS(r, r', a): If we detect a new conflict between r and r', we add (r', a) to Q. Here, we need to account for the following two cases: if $s_a^r < s_a^{r'}$, changes induced in r' do not affect r. It suffices to push (r', a) into Q. However, if $s_a^{r'} < s_a^r$, changes in r' affect r, which invalidates preceding updates made to r. In this case, we additionally push (r, a) back to Q to ensure valid arc entry time updates upon the algorithm's termination.

3.4. Online algorithm

While we discussed our matheuristic for a static problem setting, we can straightforwardly apply it to an online problem setting where trips enter the system over time, e.g., following a point process. In this context, we focus on batched decision-making, which allows us to apply our matheuristic in a rolling-horizon fashion.

Specifically, we divide the static time horizon into $n \in \{1, \dots, N\}$ distinct epochs, each spanning a fixed time interval of size Δt. Before a new epoch n starts, we then take a staggering decision for all requests that entered the system after the last decision taken, collected in a set $\mathcal{R}^n$, formally:

$$\mathcal{R}^n := \{r \in \mathcal{R} \mid (n-1) \cdot \Delta t \le e^r < n \cdot \Delta t\}. \qquad \forall n \in \{1, \dots, N\}$$

We can then compute a solution π^n for the respective batch of requests by running our matheuristic with a suitable time limit η. Algorithm 5 shows how we apply our matheuristic in such a rolling horizon setting: we iterate over all epochs (l. 2), each time focusing on the respective request set $\mathcal{R}^n$ (l. 3), and calculating the relative earliest departure times vector $\mathbf{e}^n$, and a lower bound for epoch n as described in Sections 3.2.1–3.2.3 (l. 4). For each epoch, we compute a solution π^n by running our matheuristic (l. 5). Afterward, we can fix this solution for all trips in $\mathcal{R}^n$. Still, we need to identify trips that span multiple epochs to identify all conflicts in future epochs during preprocessing. To do so, we collect such trips in a set $\mathcal{R}^T$, formally:

$$\mathcal{R}^T := \{r \in \mathcal{R}^n \mid s_{a'}^r > n \cdot \Delta t\},$$

with a' being the last arc of route p^r. Note that after the respective adjustments, trips in $\mathcal{R}^T$ can also conflict with trips being in $\mathcal{R}^n$ but not in $\mathcal{R}^T$. Maintaining those trips by incorporating them directly in $\mathcal{R}^T$ poses a computational overhead in the matheuristic, as it necessitates the inclusion of additional conflicting trips to accurately calculate their arc entry times. To mitigate this overhead, we identify all conflicts (r, r', a) in π^n, where r is part of $\mathcal{R}^T$, r' is not, and a is included in p^r. For each identified conflict, we generate an artificial trip r' with $p^{r'}$ consisting solely of the arc a, $e^{r'}$ being the arc entry time of the conflicting trip $s_a^{r'}$, $\ell^{r'}$ matching the time in which the conflicting trip completes the arc traversal, and $\bar{\sigma}^{r'}$ being zero to preclude any staggering time decisions by the algorithm. We maintain these dummy trips in a separate set $\mathcal{R}^D$ and take them into account in future epochs.

Algorithm 5 Online algorithm

Input Set of trips $\mathcal{R}$, number of epochs N, time limit η
1: $\mathcal{R}^T \leftarrow \emptyset$; $\mathcal{R}^D \leftarrow \emptyset$
2: **for** $n \in \{1, \dots, N\}$ **do**
3: $\quad \mathcal{R}^n \leftarrow$ GETTRIPSEPOCH$(\mathcal{R})$
4: $\quad \boldsymbol{e}^n, \mathrm{LB} \leftarrow$ PREPROCESS$(\mathcal{R}^n, \mathcal{R}^T, \mathcal{R}^D)$
5: $\quad \pi^n \leftarrow$ MATHEURISTIC$(\boldsymbol{e}^n, \mathrm{LB}, \eta)$
6: $\quad$ **if** $n < N$ **then**
7: $\quad\quad \mathcal{R}^T \leftarrow$ GETTRANSFERTRIPS(π^n)
8: $\quad\quad \mathcal{R}^D \leftarrow$ GETDUMMYTRIPS$(\pi^n, \mathcal{R}^T)$
9: $\boldsymbol{\pi} \leftarrow (\pi^n)_{n \in \{1,\dots,N\}}$ ▷ Vector of epoch solutions
10: $\pi \leftarrow$ GETSOLUTION$(\boldsymbol{\pi})$ ▷ Solution for every trip in $\mathcal{R}$
11: **return** π

4. Design of experiments

We used Python 3.11 and Gurobi 10.0.1 to build and solve the MILP model. To implement our matheuristic, we used C++ to implement the local search and integrated it into the Python framework via pybind11 (Jakob et al., 2017). We performed all experiments on an Intel(R) i9-9900 CPU, 3.1 GHz, with 56 GB of RAM.

To create a realistic case study, we use the New York taxi data set (NYC Taxi & Limousine Commission, 2015) and focus on the area of Manhattan in New York City.

Road network. We extract the Manhattan road network from the OpenStreetMap dataset to obtain a network with 8537 arcs and 3518 nodes (OpenStreetMap, 2024). Studying staggered routing on this raw network remains challenging as it requires checking conflicts for a large number of arcs. To decrease the number of arcs while preserving the length of the shortest paths within the network, we use contraction hierarchies (cf. Geisberger et al., 2008), a well-known reduction technique for online routing, to contract the road network graph. We employ iterative node contractions that merge arcs while preserving the shortest routes between the remaining nodes. The efficiency of this contraction hinges on the order in which nodes are processed. We first contract nodes with the smallest *edge difference* - a metric capturing the difference between the number of shortcuts added at the time of node contraction and the number of incident arcs. Here, we adopt the *lazy update* approach of Geisberger et al. (2008) to reduce edge difference recalculations and enhance efficiency.

Trip data. We obtain trips $\mathcal{R}$ from the NYC taxi dataset, collected in January 2015 (NYC Taxi & Limousine Commission, 2015), which contains taxi trips' origins, destinations, and departure times. We determine trip routes by selecting the shortest route connecting the nearest network nodes to the origin and destination with the minimum number of arcs. Trip start times e^r correspond to the taxi ride's start times provided in the dataset.

If the latest arrival times ℓ^r are too restrictive, the problem may become infeasible. A practical approach to ensure feasibility is setting ℓ^r at least as large as the arrival time in a no-staggering scenario, guaranteeing sufficient time for each trip without rescheduling. In our experiments, we implemented this by setting the maximum travel time to 125% of the nominal route travel time, creating a buffer for congestion-induced delays. This approach consistently yielded feasible instances across the congestion levels studied, ensuring all trips remained within their time windows. We calculate the maximum allowable departure time shift $\bar{\sigma}^r$ as a percentage ς^{MAX} of the total nominal travel time required to traverse the route p^r. Unless specified differently, we set this percentage to 10%.

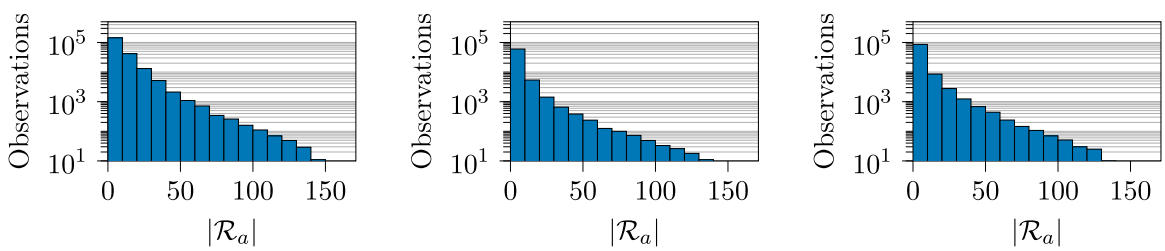
Parameterization. We set the nominal travel times τ_a assuming vehicles traveling at 20 km per hour, which approximates the average traffic speed recorded in New York City downtown in 2022 (Pishue, 2023). As we focus on the impact and control of a large AMoD fleet, we assume a baseload of traffic that is out of the operator's control and adjust residual arc capacities accordingly to account for potential congestion induced by the AMoD fleet. Specifically, we consider two scenarios: first, a LC scenario, which reflects a setting in which the exogenous traffic in the system is uncongested, and the AMoD fleet may induce congestion in case of inefficient operations. Second, a HC scenario in which the exogenous traffic already encounters congestion that might be amplified by the AMoD fleet operations. To create these scenarios, we define the arc travel time functions as piecewise linear

Table 1
Network overview for LC and HC scenarios.

	LC			HC		
	Min	Max	Avg	Min	Max	Avg
$\lvert\mathcal{A}\rvert$	7822	8384	7967	7822	8384	7972
$\lvert\mathcal{V}\rvert$	2430	2655	2487	2430	2655	2489
$\lvert\mathcal{A}\rvert^{\mathrm{m}}$	22 986	26 115	24 461	19 704	23 355	21 136
$\lvert\hat{\mathcal{A}}\rvert^{\mathrm{m}}$	1996	2604	2288	3159	3575	3351

(a) LC Scenario (b) HC Scenario

Fig. 4. Total delay in the uncontrolled solutions for both LC and HC scenarios.



(a) Undivided sets (b) LC Scenario (c) HC Scenario

Fig. 5. Trip set sizes utilizing arcs before and after processing in the LC and HC scenarios.

functions with three non-flat segments. Residual free-flow capacities are calibrated to allow one AMoD trip every 20 s in the LC scenario and every 40 s in the HC scenario, setting the first breakpoints as $\hat{f}_a^1 = \tau_a/20$ and $\hat{f}_a^1 = \tau_a/40$, rounded to the nearest integer. The remaining breakpoints are $\hat{f}_a^2 = 1.8 \cdot \hat{f}_a^1$ and $\hat{f}_a^3 = 2 \cdot \hat{f}_a^1$. The slopes are defined as $\mu_a^1 = \frac{0.5 \cdot \tau_a}{\hat{f}_a^1}$, $\mu_a^2 = \frac{0.9 \cdot \tau_a}{\hat{f}_a^1}$, and $\mu_a^3 = \frac{1.1 \cdot \tau_a}{\hat{f}_a^1}$. Appendix D provides a visualization of the breakpoints and slopes, along with a brief discussion of our choice of parameters, justified by their approximation accuracy.

Instances. We generated instances by sampling 5000 trips between 4:00 and 5:00 p.m. over 31 days, sorting them by start time. To evaluate the complexity of these instances, we computed the uncontrolled solution in both LC and HC scenarios, measuring the total delay (see Fig. 4). Notably, the total delay in HC scenarios is approximately 10 times higher than in LC scenarios. In this preliminary stage, we also assessed the impact of graph-to-multigraph processing and the trip set splitting procedure described in Sections 3.2.2–3.2.3. Table 1 provides an overview of the key network characteristics that influence instance complexity. The average number of arcs $|\mathcal{A}|$ over which trips transit is approximately 8000, while the average number of nodes $|\mathcal{V}|$ is approximately 2500. The processed multigraph features a high number of total arcs, $|\mathcal{A}|^{\mathrm{m}}$, with the LC scenario displaying a higher average. This is attributed to fewer overlapping time windows in the LC scenario, enabling a finer splitting of trips across multiple parallel arcs. Conversely, the number of conflicting arcs, $|\hat{\mathcal{A}}|^{\mathrm{m}}$, is significantly larger in HC scenarios, reflecting the greater congestion levels in the HC environment.

To further evaluate the trip set splitting procedure outlined in Section 3.2.2, Fig. 5 shows the distribution of set sizes $|\mathcal{R}_a|$ before and after partitioning in both LC and HC scenarios. The partitioning method is particularly effective in reducing large conflicting sets, with a more pronounced impact in the LC scenario due to lower arc utilization and larger gaps between trips. However, the method also significantly benefits the HC scenario by alleviating congestion on heavily utilized arcs, thus improving the overall computational efficiency.

5. Results

In the following, we discuss the findings of our numerical studies. First, we analyze our matheuristic performance for both an idealized offline setting as well as a myopic online setting in Section 5.1. We then proceed with a fine-grained analysis in Section 5.2 before discussing the trade-off between congestion-related delay reduction and trip departure time shifts in Section 5.3.

5.1. Algorithmic performance

Fig. 6 shows the distribution of the delay reduction Θ obtained when using our matheuristic compared to the uncontrolled setting for both the LC and HC scenario. We report the delay reduction obtained when using our matheuristic in a full-information offline scenario (OFF) to obtain an upper bound on the staggered routing performance. Additionally, we report the delay reduction obtained when using our matheuristic in a rolling-horizon fashion in an online scenario (ON) to quantify the benefit of staggering in a real-world setting. For further technical discussions, we refer the readers to Appendix E. We used a time limit of one hour to compute the offline solutions while limiting the computation of the rolling horizon solutions to six minutes, which equals the length of an epoch. Focusing on the offline delay reduction, we observe a median delay reduction of 98% and a minimum reduction of 85% for the LC scenario, which shows that our matheuristic allows us to mitigate at minimum 85% of the fleet-induced congestion in an initially uncongested system. For the HC scenario, we observe a delay reduction within 30% and 85%, with a median reduction of 60%, which shows that our matheuristic effectively reduces fleet-induced congestion even in a setting where the system is already congested by exogenous traffic.

Result 1 (*Offline Performance*)**.** In a full information setting, staggered routing allows for an average delay reduction of 98% and 60%, respectively, for an LC and HC scenario.

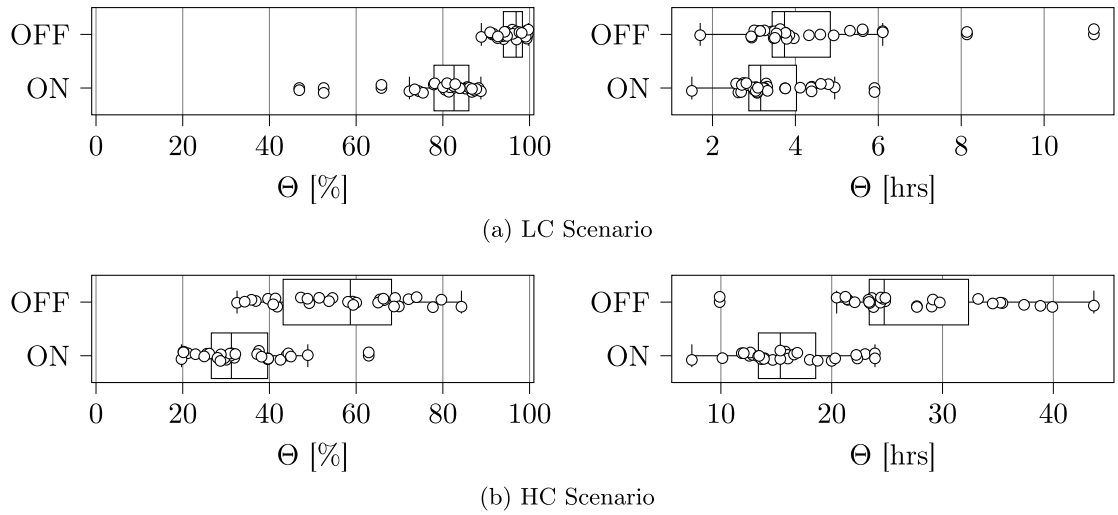


Fig. 6. Distribution of the relative and absolute delay reduction Θ over all instances for a full information offline scenario (OFF) and a rolling-horizon online scenario (ON).

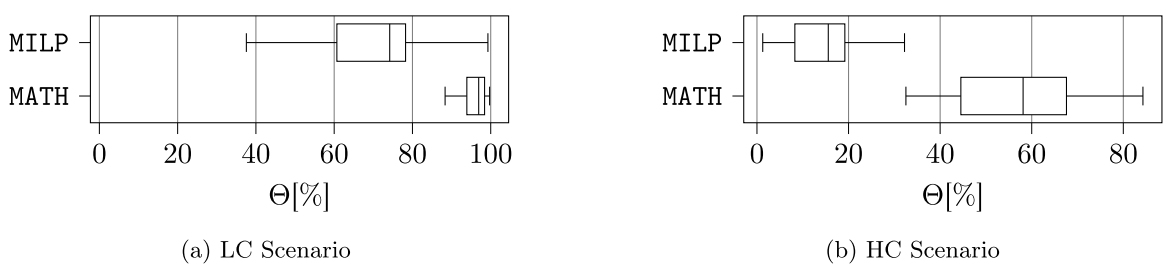


Fig. 7. Comparison of the absolute delay reductions Θ achieved by the MILP formulation alone and the full algorithm (MATH) for the LC and the HC scenario.

Focusing on the improvement potential in an online decision-making setting, we observe an average delay improvement of 82% and 30% for the LC and HC scenario when applying our matheuristic in a naive rolling horizon fashion. This underscores that even a simple, temporally local staggering of trips can significantly reduce congestion-related delays, especially in low-congestion scenarios. At the same time, it points to the potential of developing a more sophisticated prescriptive online algorithm that might obtain a performance close to the full information bound, particularly in high-congestion environments.

Result 2 (*Online Performance*)**.** In a rolling horizon setting, staggered routing allows for an average delay reduction of 82% and 30%, respectively, for an LC and HC scenario.

To understand how the local search procedure enhances the search for improved solutions, we conducted a comparative analysis between our matheuristic (MATH) and the plain MILP with a time limit set to one hour. Fig. 7 summarizes the absolute delay reductions Θ obtained by the two approaches in both the LC and HC scenarios. For the LC scenario, the performance of MATH shows a noticeable improvement compared to the MILP, achieving higher delay reductions with more consistent results across the instances. In the HC scenario, the advantage of MATH is even more pronounced, as it significantly outperforms the MILP by achieving greater delay reductions while exhibiting a wider variability in performance. These results highlight the effectiveness of the matheuristic, particularly in high-congestion settings, where it leverages its local search capabilities to explore better solutions under similar computational budgets.

5.2. Fine-grained analysis

To deepen our understanding of the impact of staggered routing, Fig. 8 shows a distribution over arcs that exhibit a certain cumulative delay within the LC or HC scenario. We detail this distribution for the arc-based delay resulting in an uncontrolled setting (UNC) in which no staggering is applied, when solving the offline full information setting (OFF), and when solving an online rolling horizon setting (ON). Note that we exclude arcs that do not exhibit any congestion in either of these settings to focus the distributions on congestion effects. Fig. 9 complements this analysis by showing the spatial distribution of trips incurring delay on certain arcs.

Clearly, the HC scenario exhibits a significantly higher cumulative delay on arcs as the existing exogenous congestion in the system reinforces the congestion induced by the AMoD fleet. However, we observe in both the LC and the HC scenarios that staggering trip departures allow us to mitigate a significant share of the congestion induced by the AMoD fleet. Specifically, our matheuristic completely eliminates congestion for approximately 1100 and 1500 arcs in the LC and HC scenarios in the offline setting, and for 800 and 1000 arcs in the online setting. Interestingly, the differences between the offline and online settings arise for the remaining mildly congested arcs for which the online algorithm does not succeed in anticipating temporal interdependencies.

Fig. 9 shows the spatial distribution of the number of trips per arc that experience delay in the respective settings. As can be seen, only a few local congestion effects remain in the LC scenario when comparing the offline and online solutions after staggering to the uncontrolled setting. Contrarily, some main roads exhibit a decreased but still significant congestion in the HC scenario.

Result 3 (*Congestion Reduction*)**.** Staggered routing increases the number of uncongested arcs, respectively, for LC and HC scenarios by 1131 and 1472 when comparing the full information solutions and 773 and 957 when comparing the naive online solutions against the uncontrolled solutions.

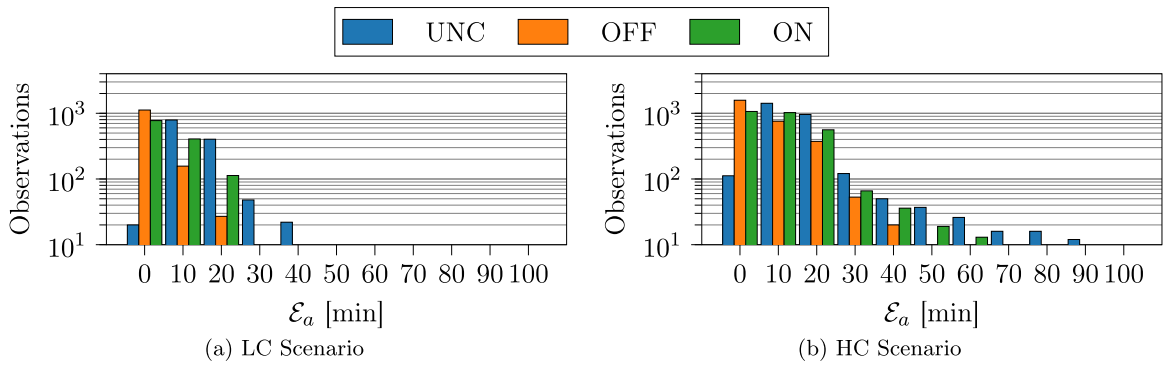


Fig. 8. Distribution of the arc-based cumulative delay ($\mathcal{E}_a$) over all instances for the uncontrolled (UNC), offline full information (OFF), and online rolling horizon (ON) settings.

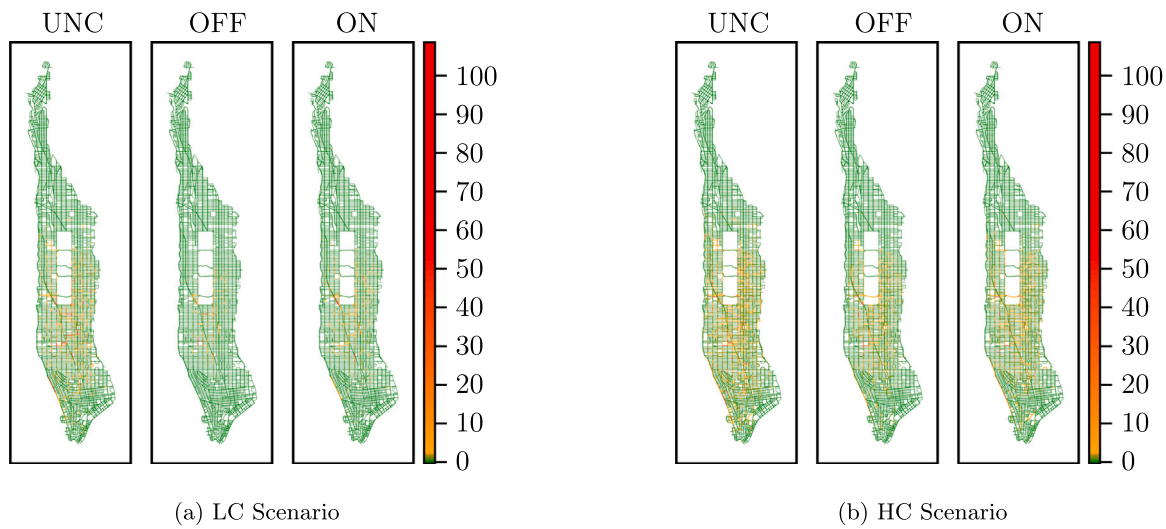


Fig. 9. Spatial distribution of the number of trips per arc that experience congestion (Ω_a) in the uncontrolled (UNC), offline (OFF), and online (ON) settings for both the LC and HC scenarios.

5.3. Sensitivity analyses

Clearly, the improvement potential of staggered routing depends on the amount of staggering allowed for each trip and constitutes a trade-off between passenger convenience, i.e., modifying a customer's departure time up to a certain threshold and obtaining a system optimum from a total congestion perspective. For this analysis, we focus on the full information setting for one single instance to discuss the maximum improvement potential of the respective trade-off. To analyze this trade-off, we ignore our basic setting and vary the maximum amount a trip's departure can be staggered within $\varsigma^{\text{MAX}} \in \{0\%, \dots, 22.5\%\}$ of its nominal travel time with a step width of 2.5 percentage points. Here, $\varsigma^{\text{MAX}} = 0\%$ equals the uncontrolled setting discussed in the preceding subsections.

Fig. 10 highlights the resulting trade-off by showing the dependency of the total accumulated delay $Z(\pi)$ and the distribution of realized trip departure shifts σ^r in dependence of the maximum possible trip departure shift ς^{MAX}, which indicates the maximum possible departure shift relative to each trip's length. We report results for both the LC and HC scenarios. As can be seen, we observe diminishing returns for delay reduction when increasing ς^{MAX} in both scenarios but at different scales. While it is possible to entirely mitigate the average cumulative delay in the LC scenario with any $\varsigma^{\text{MAX}} \geq 7.5\%$, the HC scenario shows a plateau of an average cumulative delay of around 50 h for $\varsigma^{\text{MAX}} \geq 10.0\%$. Remarkably, mitigating the entire delay in the LC scenario with maximum realized departure time shifts below two minutes is possible when choosing $\varsigma^{\text{MAX}} = 7.5\%$. Choosing a higher ς^{MAX} solely leads to higher departure time shifts. In the HC scenario, we observe increasing realized departure time shifts when increasing ς^{MAX} while yielding a decreasing overall delay in the system. Surprisingly, the realized departure time shifts remain mostly below six minutes even for high ς^{MAX}; only two outliers exhibit a shift of six minutes.

Result 4 (*LC Scenario Trade Off*)**.** In the LC scenario, a maximum trip departure shift of 7.5% relative to each trip's length allows to mitigate delays induced by the AMoD fleet entirely at the price of shifting any trip departure by at most two minutes.

Result 5 (*HC Scenario Trade Off*)**.** In the HC scenario, we observe a decreasing cumulative delay at the price of increasing realized trip departure times when increasing ς^{MAX}. Still, the realized trip departure times remain, except from two outliers at most six minutes even for high ς^{MAX}.

In conclusion, our results show that there is a trade-off between allowing higher maximum and realized departure time shifts and reducing delay in the system. However, a significant delay reduction can be obtained by accepting reasonable departure shifts between two and six minutes for both a system that comprises solely fleet-induced congestion (LC scenario) and a system that is already congested and incurs additional congestion induced by the AMoD fleet (HC scenario). In this context, it is important to recognize that while adjusting the departure time of a trip might not be preferable for a passenger, it nevertheless does not compromise the passenger's primary objective of reaching their destination punctually. This is because, independently of the ς^{MAX} parameterization, alterations to the departure time are

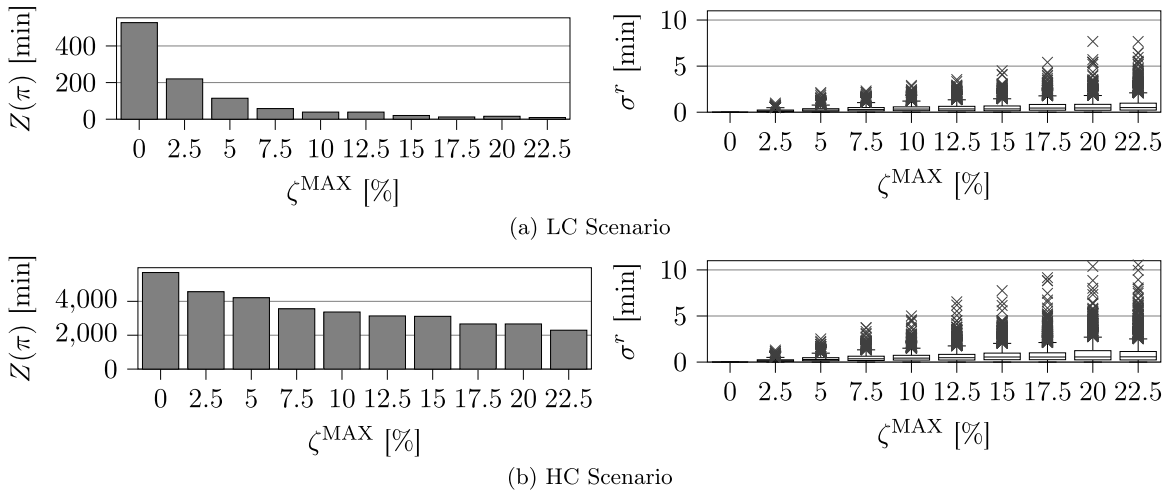


Fig. 10. Trade-off between realized trip departure time shifts σ^r, the total system delay $Z(\pi)$, and the maximum possible shift of a trip's departure relative to its length ς^{MAX} for both the LC and HC scenarios.

only permissible as long as the trip finishes within the designated time window.

6. Conclusion

We studied the impact of staggered routing, i.e., delaying the departure of trips as long as a trip's prefixed arrival time will not be exceeded to reduce local congestion bottlenecks when operating an AMoD fleet. We formalized the underlying planning problem in a full information offline setting as a MILP. We leveraged this MILP to devise an efficient matheuristic that can be used to solve large-scale full information offline instances, while at the same time being amenable to be used in a rolling-horizon online setting. We applied this matheuristic to a real-world case study for the Manhattan area in New York City. We compared the impact of staggered routing for both a full-information bound computed in an idealized offline setting and for the solution obtained in a rolling-horizon online setting against an uncontrolled case in which staggering is not allowed. We demonstrate that our matheuristic outperforms a plain MILP formulation in both low- and high-congestion scenarios. Additionally, we show that our approach can efficiently handle multi-piece, piecewise linear approximations of polynomial convex delay functions while maintaining solvability.

Our results show that in low-congestion scenarios, i.e., scenarios in which only the AMoD fleet induces congestion but the system without the fleet would remain uncongested, staggering trip departures allow to mitigate on average 98% of the induced congestion in a full information setting. While not reaching this full information bound, our naive rolling horizon approach still allows us to reduce 82% of the induced congestion in the LC scenario. In high-congestion scenarios, i.e., scenarios in which the system already encounters exogenous congestion and the AMoD fleet induces further congestion, we observe an average reduction of 60% as the full information bound and an average reduction of 30% in our online setting. Surprisingly, we show that these reductions can be reached by shifting trip departures at a maximum of two minutes in the LC scenario and by six minutes in the HC scenario, still preserving each trip's desired arrival time.

Our work presents a first step towards leveraging staggered routing in AMoD systems to reduce congestion. Specifically, we anticipate two fruitful avenues for future research. First, focusing on efficient solution methodologies that solve large-scale instances with high solution quality. Our paper represents a first step in this direction. Still, one may improve upon our results by finding better lower bounds and further enhancing algorithmic components tailored to the respective problem setting. Second, developing prescriptive online algorithms to apply staggered routing in practice. Our myopic rolling-horizon implementation already shows a significant improvement potential, giving hope to achieve even better performance with a more sophisticated online algorithm incorporating a learning-based prescriptive element. Furthermore, our analysis highlights the critical balance between passenger convenience and system-wide congestion mitigation. While short staggering times can significantly reduce congestion, practical implementation must account for passenger acceptance and market adoption. Future research should explore user behavior dynamics and investigate how staggered routing interacts with other congestion mitigation strategies, such as fare adjustments or congestion pricing.

Finally, integrating staggering directly into the routing decision process is a natural extension of our work, allowing for a more adaptive and anticipatory vehicle allocation strategy. By dynamically adjusting departures and routes in response to congestion patterns, this approach enhances system efficiency and improves overall network performance.

CRediT authorship contribution statement

Antonio Coppola: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Gerhard Hiermann:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization. **Dario Paccagnan:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization. **Maximilian Schiffer:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

Acknowledgments

The work of Antonio Coppola, Gerhard Hiermann, and Maximilian Schiffer was supported by the Deutsche Forschungsgemeinschaft (DFG) - Project number 449261765 (BalSAM). The work of Dario Paccagnan was supported by the EPSRC grant EP/Y001001/1, funded by the International Science Partnerships Fund (ISPF) and UKRI.

Appendix A. Hardness proof

To prove the hardness of the staggered routing problem as defined in Section 2, we utilize the hardness result for the three machines job shop scheduling problem with unit processing times and no-wait constraints (J3-UPT-NWT) with makespan (the maximum completion time of a job) minimization as objective, which is NP-hard (Sriskandarajah & Ladet, 1986).

Theorem 1. *The staggered routing problem is NP-hard.*

Proof. We begin by defining instances for J3-UPT-NWT and for the staggered routing problem:

Definition 1. Let I be an instance of the J3-UPT-NWT with $m = 3$ machines and jobs J_i, with $i = 1, \dots, n$, where each job comprises $N(J_i)$ operations that each require one unit of processing time and must be executed continuously from start to finish. The operations are allocated to specific machines, and the order in which machines are visited may vary among different jobs. Each machine can process a maximum of one job at a time. The J3-UPT-NWT seeks for a job schedule S, i.e., the starting times $s(J_i)$ of the first operation of each job J_i, that minimizes the schedule's makespan D.

Definition 2. Let I' be an instance of the staggered routing problem with n trips as defined in Section 2. Trips must proceed along their routes without interruption within designated time windows. The travel time for a trip along an arc is influenced by the number of trips on the arc at the specific time the trip enters the arc. If the number of trips on the arc falls below a predefined arc-specific threshold capacity, the trip proceeds at the nominal travel time designated for that arc. Conversely, exceeding this threshold triggers a delay, as detailed in Eq. (2.1). The objective of the staggered routing problem is to find the trip departures s_a^r that minimize the total travel time.

Given these definitions, we prove the hardness of the staggered routing problem by contradiction.

Assumption 1. It exists an algorithm $\mathscr{A}$ that finds a feasible solution for every instance of the staggered routing problem in polynomial time.

Step 1: Consider an instance I' that comprises n trips traversing $m = 3$ arcs, where at least two trips have different routes. The earliest possible departure time for trips is set to zero, with no restrictions on the staggering of departures. All trips must finish by time D. Each arc has a nominal travel time and a threshold capacity of one; exceeding this threshold on any arc introduces an infinite delay.

Step 2: Instance I' can be transformed in polynomial time to an instance I as follows: the route of each trip corresponds to a job to be executed without interruptions from start to finish. The act of traversing the arcs within a trip's route mirrors the operations within a job. These operations are executed by a dedicated machine (an arc), each with a unit processing time (equivalent to nominal travel time), and concurrent processing of operations on the same machine is prohibited (otherwise, one incurs infinite delay). The latest arrival times translate to the schedule's makespan under consideration. Finally, the departure times from the arcs on a route correspond to the scheduling times of the job operations.

Step 3: Observe that selecting the job's starting times in instance I to decide whether a schedule S of makespan of D exists is equivalent to finding feasible trip departure times in instance I', such that we obtain a polynomial-time transformation from the respective staggered routing problem instance to an equivalent J3-UPT-NWT instance. Then, if $\mathscr{A}$ can find a feasible solution in polynomial time for I', it can also find a feasible solution for I within polynomial time. However, finding a solution for I in polynomial time is impossible unless P = NP, contradicting Assumption 1. Accordingly, there exists at least one instance of the staggered routing problem that cannot be solved in polynomial time. □

Appendix B. Table of variables in the MILP

See Table 2.

Table 2
Table of variables in the MILP.

Variable	Description
$\mathcal{R}$	Set of all trips $r \in \mathcal{R}$
$\mathcal{A}$	Set of all arcs $a \in \mathcal{A}$
$\mathcal{R}_a$	Subset of $\mathcal{R}$, set of trips traversing arc a
p^r	Route of trip r
$\delta^r(a)$	Arc succeeding a on route p^r
$\underline{a}^r$	First arc of p^r
$\bar{a}^r$	Last arc of p^r
x_a^r	Delay on arc a for trip r
μ_a^k	Slope of the kth segment of arc a travel time function.
τ_a	Nominal travel time for arc a
f_a^r	Number of vehicles encountered by trip r on arc a
$\hat{f}_a^k$	Flow value at which the kth segment of arc a travel time function begins.
s_a^r	Departure time of trip r on arc a
e^r	Earliest possible departure time for trip r
ℓ^r	Latest possible arrival time for trip r
$\bar{\sigma}^r$	Maximum staggering applicable to trip's r departure
$\alpha_a^{r,r'}, \beta_a^{r,r'}, \gamma_a^{r,r'}$	Logical variables checking concurrent presence of trips (r, r') on arc a
ε	Small constant
M_i	Large constant

Table 3
Current time windows on a_1.

Trip	Departure	Arrival
r	$[0, 1]$	$[1, 3]$
r'	0.5	$[1.5, 2.5]$

Table 4
Current time windows on a_2.

Trip	Departure	Arrival
r	$[1, 3]$	$[2, 5]$
r'	$[1.5, 2.5]$	$[2.5, 4.5]$

Table 5
Time windows on a_1 for Case 1.

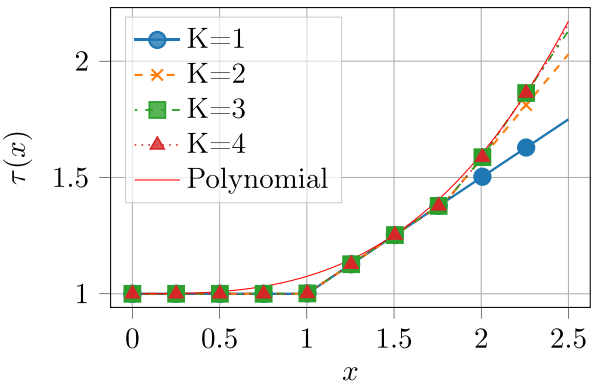
Trip	Departure	Arrival
r	$[0, 0.5]$	$[1, 1.5]$
r'	0.5	2.5

Appendix C. Example of tighter time windows computation

As discussed in Section 3.2.1, examining one arc at a time inevitably overestimates the size of the time windows. This occurs because the current method ignores the travel continuity constraints that trips must respect when transitioning from one arc to the next. We clarify this limitation with the following example. Consider two consecutive arcs, a_1 and a_2, each with unit free-flow travel time. A conflict on either arc causes a unit delay for the trip behind. We analyze two trips, r and r', both traversing a_1 and a_2. Trip r can start on a_1 within $[0, 1]$, while trip r' is fixed to start at 0.5. Using the current procedure, which disregards upstream arcs, we obtain the time windows reported respectively in Tables 3–4.

We now recompute the time windows by incorporating upstream constraints on a_1. This requires distinguishing two cases:

Case 1: Trip r starts on a_1 within $[0, 0.5]$. Time windows reported respectively in Tables 5–6.
Case 2: Trip r starts on a_1 within $(0.5, 1]$. Time windows reported in Tables 7–8.
Combining both cases, we obtain the following tighter arrival time windows for a_2: $[2, 4]$ for trip r and $[2.5, 3.5]$ for trip r', compared to the previously computed intervals of $[2, 5]$ and $[2.5, 4.5]$, respectively.

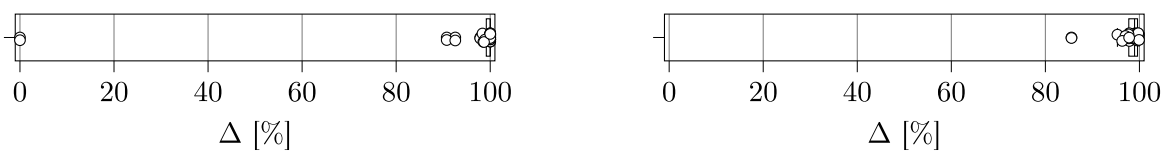


(a) Piecewise linear functions vs. polynomial.

K	MAE	RMSE
1	0.068	0.12
2	0.029	0.041
3	0.019	0.027
4	0.018	0.026

(b) Error metrics for piecewise functions.

Fig. 11. Comparison of piecewise linear functions and their error metrics.



(a) LC scenario (b) HC scenario

Fig. 12. Distribution of the optimality gap Δ [%] over all instances.

Table 6
Time windows on a_2 for Case 1.

Trip	Departure	Arrival
r	$[1, 1.5]$	$[2, 2.5]$
r'	2.5	3.5

Table 7
Time windows on a_1 for Case 2.

Trip	Departure	Arrival
r	$(0.5, 1]$	$[2.5, 3]$
r'	0.5	1.5

Table 8
Time windows on a_2 for Case 2.

Trip	Departure	Arrival
r	$[2.5, 3]$	$[3.5, 4]$
r'	1.5	2.5

Appendix D. Travel time function parameterization

In Fig. 11, we compare piecewise linear functions with K non-flat segments to assess their approximation error relative to a polynomial function modeling travel times under varying congestion levels. Here, x represents the flow to arc capacity ratio, while $\tau(x)$ denotes the corresponding travel time multiplier. Each function is constructed by sequentially introducing segments k at breakpoints $\hat{f}^k$ with progressively increasing slope μ^k. The selected slopes and breakpoints are $(\mu^1, \hat{f}^1) = (0.5, 1)$, $(\mu^2, \hat{f}^2) = (0.9, 1.8)$, $(\mu^3, \hat{f}^3) = (1.1, 2)$, and $(\mu^4, \hat{f}^4) = (1.4, 2.4)$. To assess the approximation error relative to the polynomial function, we report the mean absolute error (MAE) and root mean square error (RMSE). The results indicate diminishing marginal improvements as additional segments are introduced. Beyond $K = 3$, the reduction in the approximation error becomes negligible, such that choosing an approximation with three segments maintains a reasonable trade-off between accuracy and computational complexity.

Appendix E. Optimality gaps

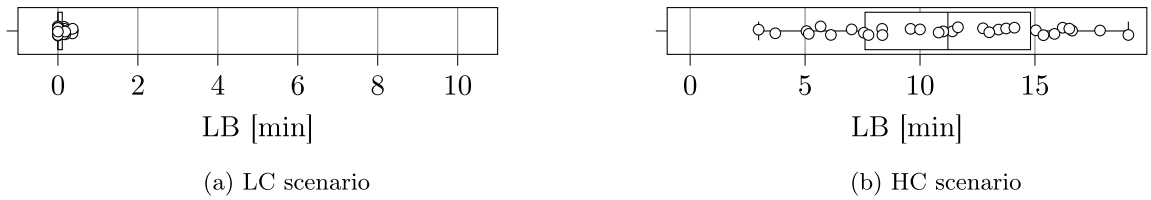
Figs. 12–14 show the distribution of the optimality gap (Fig. 12), the absolute value of the lower bound (Fig. 13), and the distance between the upper bound and the lower bound (Fig. 14) for both the LC scenario and the HC scenario, obtained when running our matheuristic with a time limit of one hour.

Focusing on the relative optimality gaps in Fig. 12, we observe a bipolar distribution in which optimality gaps accumulate either around 0% or around 100% for the LC scenario. For the HC scenario, most relative optimality gaps range between 90% and 100%. At first sight, these optimality gaps look anything but satisfying in terms of algorithmic performance, but unravel as follows at second sight: as our objective is to minimize the total congestion-related delay in the system, a trivial lower bound on our objective value is an objective value of zero. Clearly, our problem bears a huge degree of symmetry, which leads to weak lower bounds that cannot be significantly improved in our MILP formulation, see Fig. 13. In such cases, an absolute distance of even a few minutes of delay left in the system can create a large optimality gap.

To support our claim that our matheuristic finds good solutions, we compare these optimality gaps to the delay reductions shown in Fig. 6. In fact, a comparison to the uncontrolled solution provides more insights compared to a lower bound comparison: relating Figs. 6 to 12, it becomes obvious that the maximum improvement potential to reduce delay remains at 20% and 60% respectively for the LC and HC scenarios, although the optimality gap distribution shows values up to 100%.

Remark 3 (*Lower Bound*)**.** When minimizing the total congestion-related delay in a staggered routing setting, the relative optimality gap is an uninformative indicator of solution quality due to the nature of the objective function and the problem's weak lower bounds.

In this context, Fig. 14 reports the difference between the solutions found and the lower bound, indicating that our matheuristic founds good solutions despite large optimality gaps. Specifically, we observe an average distance between the upper (UB) and lower (LB) bound



(a) LC scenario (b) HC scenario

Fig. 13. Distribution of the absolute value of the lower bound LB [min] over all instances.

(a) LC scenario (b) HC scenario

Fig. 14. Distribution of the distance between the upper and lower bound UB-LB [min] over all instances.

of 10 min and 20 h for the LC and HC scenarios, respectively. As the instances exhibit respectively an average delay of 4 h and 45 h in an uncontrolled setting, the absolute distance between the upper and lower bounds appears relatively small for the LC scenario and still satisfactory for the HC scenario, indicating that our matheuristic effectively reduces congestion via staggering.

References

Adacher, L., & Tiriolo, M. (2018). A macroscopic model with the advantages of microscopic model: A review of cell transmission model's extensions for urban traffic networks. *Simulation Modelling Practice and Theory*, *86*, 102–119.

Agarwal, S., Mani, D., & Telang, R. (2023). The impact of ride-hailing services on congestion: Evidence from Indian cities. *Manufacturing & Service Operations Management*, *25*(3), 862–883.

Alonso-Mora, J., Wallar, A., & Rus, D. (2017). Predictive routing for autonomous mobility-on-demand systems with ride-sharing. In *International conference on intelligent robots and systems (IROS)*. IEEE.

Arnott, R., de Palma, A., & Lindsey, R. (1990). Economics of a bottleneck. *Journal of Urban Economics*, *27*(1), 111–130.

Bang, H., & Malikopoulos, A. A. (2022). Congestion-aware routing, rebalancing, and charging scheduling for electric autonomous mobility-on-demand system. In *2022 American control conference* (pp. 3152–3157).

Baptiste, P., Le Pape, C., & Nuijten, W. (2001). *Constraint-based scheduling: applying constraint programming to scheduling problems*: *vol. 39*, Springer Science & Business Media.

Bartusch, M., Möhring, R. H., & Radermacher, F. J. (1988). Scheduling project networks with resource constraints and time windows. *Annals of Operations Research*, *16*, 199–240.

Battifarano, M., & Qian, S. (2023). The impact of optimized fleets in transportation networks. *Transportation Science*, *57*(4), 1047–1068.

Bennett, A. (2006). *Lagrangian fluid dynamics* (p. 286). Cambridge, UK: Cambridge University Press.

Boyd, S., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press.

Chabini, I. (1998). Discrete dynamic shortest path problems in transportation applications: Complexity and algorithms with optimal run time. *Transportation Research Record*, *1645*(1), 170–175.

Daganzo, C. F. (2007). Urban gridlock: Macroscopic modeling and mitigation approaches. *Transportation Research Part B: Methodological*, *41*(1), 49–62.

Enders, T., Harrison, J., Pavone, M., & Schiffer, M. (2023). Hybrid multi-agent deep reinforcement learning for autonomous mobility on demand systems. http://dx.doi.org/10.48550/arXiv.2212.07313, arXiv:2212.07313 [cs.LG].

Estandia, A., Schiffer, M., Rossi, F., Luke, J., Kara, E. C., Rajagopal, R., & Pavone, M. (2021). On the interaction between autonomous mobility on demand systems and power distribution networks—An optimal power flow approach. *IEEE Transactions on Control of Network Systems*, *8*(3), 1163–1176.

Gammelli, D., Yang, K., Harrison, J., Rodrigues, F., Pereira, F. C., & Pavone, M. (2021). Graph neural network reinforcement learning for autonomous mobility-on-demand systems. In *2021 60th IEEE conference on decision and control* (pp. 2996–3003). IEEE.

Geisberger, R., Sanders, P., Schultes, D., & Delling, D. (2008). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In C. C. McGeoch (Ed.), *Experimental algorithms* (pp. 319–333). Springer Berlin Heidelberg.

Iglesias, R., Rossi, F., Wang, K., Hallac, D., Leskovec, J., & Pavone, M. (2018). Data-driven model predictive control of autonomous mobility-on-demand systems. In *2018 IEEE international conference on robotics and automation* (pp. 1—-7). IEEE Press.

Jahn, O., Möhring, R. H., Schulz, A. S., & Stier-Moses, N. E. (2005). System-optimal routing of traffic flows with user constraints in networks with congestion. *Operations Research*, *53*(4), 600–616.

Jakob, W., Rhinelander, J., & Moldovan, D. (2017). pybind11 – seamless operability between c++11 and python. URL https://github.com/pybind/pybind11.

Jalota, D., Paccagnan, D., Schiffer, M., & Pavone, M. (2023). Online routing over parallel networks: Deterministic limits and data-driven enhancements. *INFORMS Journal on Computing*, *35*(3), 560–577.

Jiao, Y., Tang, X., Qin, Z. T., Li, S., Zhang, F., Zhu, H., & Ye, J. (2021). Real-world ride-hailing vehicle repositioning using deep reinforcement learning. *Transportation Research Part C: Emerging Technologies*, *130*, Article 103289.

Jungel, K., Parmentier, A., Schiffer, M., & Vidal, T. (2023). Learning-based online optimization for autonomous mobility-on-demand fleet control. http://dx.doi.org/10.48550/arXiv.2302.03963, arXiv:2302.03963 [math.OC].

Köhler, E., Langkau, K., & Skutella, M. (2002). Time-expanded graphs for flow-dependent transit times. In *Algorithms—ESA 2002: 10th annual European symposium rome, Italy, September 17–21, 2002 proceedings 10* (pp. 599–611). Springer.

Kumarage, S., Yildirimoglu, M., Ramezani, M., & Zheng, Z. (2021). Schedule-constrained demand management in two-region urban networks. *Transportation Science*, *55*(4), 857–882. http://dx.doi.org/10.1287/trsc.2021.1052, arXiv:https://doi.org/10.1287/trsc.2021.1052.

Levin, M. W., Kockelman, K. M., Boyles, S. D., & Li, T. (2017). A general framework for modeling shared autonomous vehicles with dynamic network-loading and dynamic ride-sharing application. *Computers, Environment and Urban Systems*, *64*, 373–383.

Levy, J. I., Buonocore, J. J., & Von Stackelberg, K. (2010). Evaluation of the public health impacts of traffic congestion: a health risk assessment. *Environmental Health*, *9*, 1–12.

Li, M., Qin, Z., Jiao, Y., Yang, Y., Wang, J., Wang, C., Wu, G., & Ye, J. (2019). Efficient ridesharing order dispatching with mean field multi-agent reinforcement learning. In *The world wide web conference* (pp. 983—-994). New York, NY, USA: Association for Computing Machinery.

Liang, E., Wen, K., Lam, W. H., Sumalee, A., & Zhong, R. (2021). An integrated reinforcement learning and centralized programming approach for online taxi dispatching. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(9), 4742–4756.

Liu, Y., & Samaranayake, S. (2022). Proactive rebalancing and speed-up techniques for on-demand high capacity ridesourcing services. *IEEE Transactions on Intelligent Transportation Systems*, *23*(2), 819–826.

Menelaou, C., Timotheou, S., Kolios, P., & Panayiotou, C. G. (2022). Joint route guidance and demand management for real-time control of multi-regional traffic networks. *IEEE Transactions on Intelligent Transportation Systems*, *23*(7), 8302–8315.

Menelaou, C., Timotheou, S., Kolios, P., & Panayiotou, C. (2024). Efficient demand management for the on-time arrival problem: A convexified multi-objective approach assuming macroscopic traffic dynamics. *European Journal of Control*, Article 101057.

Narayanan, S., Chaniotakis, E., & Antoniou, C. (2020). Shared autonomous vehicle services: A comprehensive review. *Transportation Research Part C: Emerging Technologies*, *111*, 255–293.

Neumann, K., Schwindt, C., & Zimmermann, J. (2002). *Project scheduling with time windows and scarce resources: temporal and resource-constrained project scheduling with regular and nonregular objective functions*: *vol. 508*, Springer.

NYC Taxi & Limousine Commission (2015). TLC trip record data. URL https://www1.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

OpenStreetMap 2024. URL https://www.openstreetmap.org.

Otsubo, H., & Rapoport, A. (2008). Vickrey's model of traffic congestion discretized. *Transportation Research Part B: Methodological*, *42*(10), 873–889.

Patriksson, M. (2015). *The traffic assignment problem: Models and methods*. Courier Dover Publications.

Pavone, M. (2015). Autonomous mobility-on-demand systems for future urban mobility. In *Autonomes fahren: technische, rechtliche und gesellschaftliche aspekte* (pp. 399–416). Berlin, Heidelberg: Springer Berlin Heidelberg.

Pishue, B. (2023). *2022 INRIX global traffic scorecard*: *Technical Report INRIX*, INRIX.

Ramezani, M., Haddad, J., & Geroliminis, N. (2015). Dynamics of heterogeneity in urban networks: aggregated traffic modeling and hierarchical control. *Transportation Research Part B: Methodological*, *74*, 1–19.

Rossi, F., Zhang, R., Hindy, Y., & Pavone, M. (2018). Routing autonomous vehicles in congested transportation networks: structural properties and coordination algorithms. *Autonomous Robots*, *42*(7), 1427–1442.

Roughgarden, T. (2005). *vol. 74*, *Selfish routing and the price of anarchy*. MIT Press.

Ruch, C., Lu, C., Sieber, L., & Frazzoli, E. (2020). Quantifying the efficiency of ride sharing. *IEEE Transactions on Intelligent Transportation Systems*, *22*(9), 5811–5816.

Sadeghi Eshkevari, S., Tang, X., Qin, Z., Mei, J., Zhang, C., Meng, Q., & Xu, J. (2022). Reinforcement learning in the wild: Scalable RL dispatching algorithm deployed in ridehailing marketplace. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 3838—-3848). New York, NY, USA: Association for Computing Machinery.

Salazar, M., Lanzetti, N., Rossi, F., Schiffer, M., & Pavone, M. (2019). Intermodal autonomous mobility-on-demand. *IEEE Transactions on Intelligent Transportation Systems*, *21*(9), 3946–3960.

Skordilis, E., Hou, Y., Tripp, C., Moniot, M., Graf, P., & Biagioni, D. (2021). A modular and transferable reinforcement learning framework for the fleet rebalancing problem. *IEEE Transactions on Intelligent Transportation Systems*, *23*(8), 11903–11916.

Sriskandarajah, C., & Ladet, P. (1986). Some no-wait shops scheduling problems: Complexity aspect. *European Journal of Operational Research*, *24*(3), 424–438, Flexible Manufacturing Systems.

Tang, X., Qin, Z. T., Zhang, F., Wang, Z., Xu, Z., Ma, Y., Zhu, H., & Ye, J. (2019). A deep value-network based approach for multi-driver order dispatching. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1780–1790). New York, NY, USA: Association for Computing Machinery.

Tirachini, A., & Gomez-Lobo, A. (2020). Does ride-hailing increase or decrease vehicle kilometers traveled (VKT)? A simulation approach for santiago de Chile. *International Journal of Sustainable Transportation*, *14*(3), 187–204.

Tsao, M., Iglesias, R., & Pavone, M. (2018). Stochastic model predictive control for autonomous mobility on demand. In *2018 21st international conference on intelligent transportation systems* (pp. 3941–3948).

US Bur. Public Roads (1964). *vol. 2*, *Traffic assignment manual for application with a large, high speed computer*. Washington, DC: US Gov. Print. Off..

Vickrey, W. S. (1969). Congestion theory and transport investment. *The American Economic Review*, 251–260.

Xu, Z., Li, Z., Guan, Q., Zhang, D., Li, Q., Nan, J., Liu, C., Bian, W., & Ye, J. (2018). Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 905—-913). New York, NY, USA: Association for Computing Machinery.

Yildirimoglu, M., & Ramezani, M. (2020). Demand management with limited cooperation among travellers: A doubly dynamic approach. *Transportation Research Part B: Methodological*, *132*, 267–284, 23rd International Symposium on Transportation and Traffic Theory (ISTTT 23).

Zardini, G., Lanzetti, N., Pavone, M., & Frazzoli, E. (2022). Analysis and control of autonomous mobility-on-demand systems. *Annual Review of Control, Robotics, and Autonomous Systems*, *5*(1), 633–658.

Zhang, L., Hu, T., Min, Y., Wu, G., Zhang, J., Feng, P., Gong, P., & Ye, J. (2017). A taxi order dispatch model based on combinatorial optimization. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 2151—-2159). New York, NY, USA: Association for Computing Machinery.

Zhou, M., Jin, J., Zhang, W., Qin, Z., Jiao, Y., Wang, C., Wu, G., Yu, Y., & Ye, J. (2019). Multi-agent reinforcement learning for order-dispatching via order-vehicle distribution matching. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 2645—-2653). New York, NY, USA: Association for Computing Machinery.

Ziliaskopoulos, A. K., Waller, S. T., Li, Y., & Byram, M. (2004). Large-scale dynamic traffic assignment: Implementation issues and computational analysis. *Journal of Transportation Engineering*, *130*(5), 585–593.